AGV 智能调度系统 项目学习指导书

配套课程：制造系统感知分析与决策

覆盖内容：课程任务 1 至任务 7 的完整实现原理与代码导读

技术栈：Python 3.10 · Django 5.2 · Django REST Framework · MySQL 8.0 · Vue 3 · OpenCV · scikit-learn · statsmodels

适用读者：项目小组成员，以及任何需要看懂这套代码的人

目录

这本材料怎么用 4

第 1 章　先建立整体认识 4

1.1 一句话说清楚：这个系统做什么 4
1.2 七个任务与课程要求的对应关系 5
1.3 系统总体架构 5
1.4 一次点击背后发生了什么 6
1.5 项目目录导航 7

第 2 章　把基础补回来 9

2.1 前端、后端、数据库，各干什么 9
2.2 API、REST、JSON 到底是什么 9
2.3 什么是 ORM，为什么不写 SQL 9
2.4 Python 里你一定会反复看到的写法 10
2.5 运行环境里必须知道的几件事 10

第 3 章　任务 3：地图建模与最短路径规划 12

3.1 地图是怎么变成“图”的 12
3.2 直觉：什么是最短路径 13
3.3 Dijkstra 算法：一句话说透 13
3.4 换个角度看：最短距离场 13
3.5 代码在哪里，怎么读 14
3.6 路径之外的约束检查 15
3.7 关于任务 3 的常见提问 15

第 4 章　任务 1：单台 AGV 的多任务顺序优化 16

4.1 问题到底难在哪 16
4.2 从穷举到动态规划 16
4.3 任务数超过 10 怎么办 17
4.4 用哪台车？——综合成本选车 18
4.5 运行结果长什么样 19
4.6 关于任务 1 的常见提问 20

第 5 章　任务 2：多台 AGV 联合调度 21

5.1 问题从“排序”变成了“分配 + 排序” 21
5.2 基准算法：一个讲得清的“笨办法” 21
5.3 初始解：最优插入 22
5.4 变邻域搜索（VNS）在做什么 23
5.5 多车避碰：节点时间窗预约 24
5.6 综合目标函数：算法到底在“最小化”什么 24
5.7 一个必须提前知道的坑：改进率可能显示为 0 25
5.8 关于任务 2 的常见提问 25

第 6 章　任务 4 / 5：机器视觉 26

6.1 图像在计算机里到底是什么 26

6.2 任务 4 裂纹检测的完整处理链 26
6.3 关键知识点：什么是“黑脊”，Frangi 在算什么 27
6.4 怎么判断检测结果好不好 27
6.5 任务 5 工件识别：从像素到“这是什么形状” 29
6.6 与任务书描述的差异，以及实测指标 29
6.7 关于任务 4 / 5 的常见提问 30
第 7 章　任务 6：设备状态监测 31
7.1 这是一个“分类”问题 31
7.2 为什么先要“标准化” 31
7.3 从一棵决策树到一片森林 31
7.4 模型表现怎么衡量 33
7.5 规则告警：模型之外的第二道闸 33
7.6 数据来源与已知局限 33
7.7 关于任务 6 的常见提问 34
第 8 章　任务 7：订单需求预测 35
8.1 时间序列和普通回归有什么不一样 35
8.2 三个模型分别在做什么 35
8.3 怎么判断哪个模型更好 36
8.4 数据来源与已知局限 37
8.5 关于任务 7 的常见提问 37
第 9 章　工程实现：前后端是怎么接起来的 38
9.1 Django 在这个项目里承担了什么 38
9.2 主要接口清单 38
9.3 前端：8 个页面与一个轮询 39
9.4 界面一览 40
9.5 文件上传与图片存储 42
9.6 如果要部署到服务器 42
第 10 章　测试、数据与复现 43
10.1 自动化测试覆盖了什么 43
10.2 从零复现整套环境 43
10.3 几个会让演示翻车的坑 44
第 11 章　答辩准备 45
11.1 五分钟演示脚本 45
11.2 高频问题速查 45
11.3 薄弱环节与应对原则 46
11.4 小组分工与材料清单 47
附录 A　术语速查 48
附录 B　关键代码位置索引 49
附录 C　核心指标汇总 49

这本材料怎么用

你手上这套代码已经跑起来了，七个任务的实现都写完了，测试也能全部通过。所以这本材料不再讲“怎么做出来”，而是讲“它到底在做什么、为什么这么做、每一句关键代码在哪里”。学习目标和准备答辩是同一件事的两面：真正把原理讲明白了，答辩时自然答得上；反过来，只背话术，问第二句就会露馅。

阅读顺序建议：先通读第 1 章建立整体印象，再按“第 3 章（路径）→ 第 4 章（单台调度）→ 第 5 章（多台调度）”走一遍调度主线；如果你想先看简单的，可以先读第 6、7、8 章，这三个任务的数学门槛更低。第 2 章是基础补丁，哪一段看不懂就回头翻哪一段，不必按顺序读完整章。

关于难度：材料假设你只学过 C 语言、Python、数据结构、人工智能与计算科学、高等数学，而且忘得差不多了。所有数学部分都先用一句话说清直觉，再给公式；公式看不懂时，读它上面的解释句就够了。

关于图片：文中的示意图和界面截图全部来自这个项目的真实数据与真实运行结果，不是找来的图。截图里的数字（节点数、里程、耗时、准确率）你在自己电脑上重新跑一遍就能复现。

> **提醒**　本材料是给你学习用的第三方整理稿。它不能替代你自己的理解，也不应该被当成答辩话术直接背。凡是涉及分工、工作量、个人贡献的表述，请以你和组员的真实情况为准。

第 1 章　先建立整体认识

1.1 一句话说清楚：这个系统做什么

这套系统是一个“虚拟仓库里的 AGV 调度平台”。它把一间仓库抽象成一张地图，地图上有道路和障碍，有取货点和放货点；仓库里有 4 台 AGV，也就是无人搬运小车。用户提交若干条搬运任务（把某样货物从 A 点运到 B 点），系统负责算出：用哪几台车、每台车按什么顺序做哪些任务、每一段走哪条路、大概什么时候到、会不会撞车。

在这个调度内核之外，项目还挂上了三个与智能制造相关的智能模块：用摄像头照片判断工件表面有没有裂纹、识别工件的形状与位置、根据传感器数据判断机床刀具处于正常还是磨损状态、以及根据历史订单量预测未来的订单需求。

所以整个项目其实是“1 个调度内核 + 3 个智能模块 + 1 套网页界面”，对应课程任务书里的 7 个任务。

1.2 七个任务与课程要求的对应关系

课程任务书原文只有一段话描述一个任务，下面这张表把任务书要求、本项目实现方式、代码位置三者对齐，准备答辩时可以照着这张表逐条核对。

课程任务	任务书要求（原文要点）	本项目实现	对应代码
任务 1　单台 AGV 调度决策	单台 AGV 执行多个搬运任务，确定任务执行顺序，使整个搬运过程用时最短	状态压缩动态规划求精确最优顺序；任务数超过 10 时改用最近邻 + 2-opt；同时按空驶距离、电量、载重裕量综合选车	SchedulingDecision/services.py optimize_single_solution()
任务 2　多台 AGV 调度决策	对 AGV 群组协调分配搬运任务，提升物流效率，减少整体搬运时间，减少设备使用数量	最优插入构造初始解 + 变邻域搜索联合优化“任务归属”与“车内顺序”；节点时间窗检测多车冲突并加入避碰等待	SchedulingDecision/services.py optimize_multi_solution()
任务 3　路径规划	根据起点与终点，参考地图规划行进路线，使行驶距离最短并满足其他约束	地图建模为带权无向图，用 Dijkstra 求最短路径；叠加电量约束校验与耗时、耗电估算	SchedulingDecision/services.py GraphRouter._dijkstra()
任务 4　工件表面质量检测	图像预处理、滤波增强突出裂纹特征；应用边缘检测算子识别工件与裂纹边界；对区域设定大小阈值判断裂纹	高斯滤波去噪 + 多尺度 Frangi 黑脊响应突出线状暗结构 + 自适应阈值分割 + 连通域一致性筛选（未使用边缘检测算子，原因见 6.6 节）	ManufacturingVision/services.py analyze_crack()
任务 5　物体识别	确定需要选取的目标工件，通过机器视觉算法给出工件位置信息，供机械臂抓取	双边滤波 + Otsu 阈值分割 + 形态学开闭运算 + 轮廓提取 + 多边形逼近 + 形状分类，输出包围框与中心坐标	ManufacturingVision/services.py analyze_workpiece()
任务 6　设备状态监测	对设备运行的多变量传感器数据实时监测与可视化；用机器学习算法或规则库判断状态（如磨损、正常）	StandardScaler 标准化 + 随机森林三分类（正常/磨损/故障），输出置信度与特征重要性；叠加机理阈值规则告警	EquipmentMonitoring/services.py diagnose_equipment()
任务 7　订单预测	根据历史订单预测未来一段时间的订单量，利用多种时间序列预测算法实现精准预测	移动平均、Holt-Winters、ARIMA 三模型；自动模式先回测比较 RMSE/MAPE，选优后再用完整历史预测未来	DemandForecast/services.py forecast_demand()

这张表里的每一行，都是一个可以单独答辩的小课题。如果你只想在答辩里讲一个任务，建议选任务 4（裂纹检测）或任务 1（单台调度），因为它们的原理最容易在五分钟内讲清楚，而且有直观的图和数字。

1.3 系统总体架构

这套系统是典型的“前后端分离”结构。浏览器里跑的是前端页面，它自己不存任何数据；所有数据和计算都在后端的 Django 服务里；后端算完的结果存进 MySQL 数据库，下次打开页面还能看到历史记录。三者的分工可以用下面这张图记住。

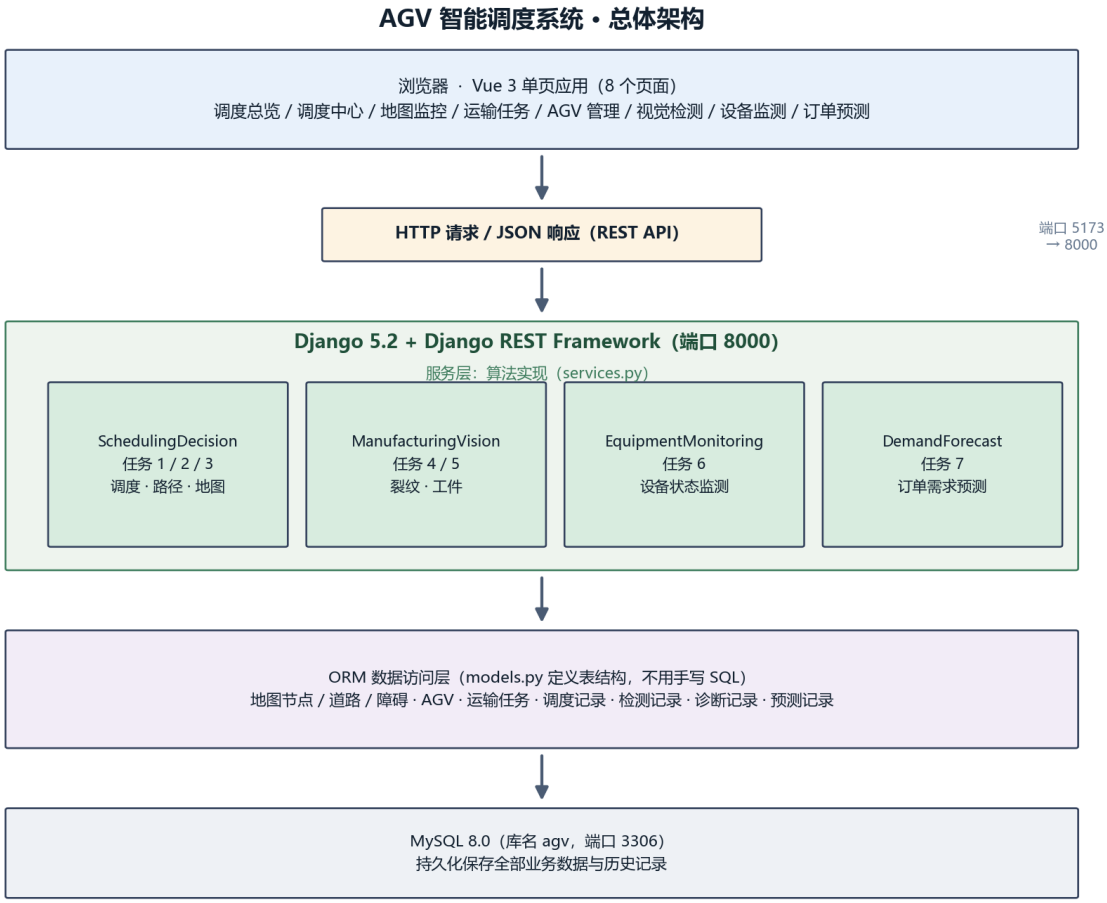


图 1　系统总体架构：前端只负责显示与收集输入，后端负责全部计算，数据库负责长期保存

把这张图从上往下读一遍，你就理解了整个项目的数据流向：用户在网页上点一个按钮 → 浏览器把请求打包成 JSON 发给后端 → 后端的某个应用执行算法 → 算法通过 ORM 读写数据库 → 结果再按 JSON 发回浏览器 → 页面刷新。

1.4 一次点击背后发生了什么

用“在地图监控页规划一条最短路径”这个具体动作，把上面的流程走一遍。这个过程在软件开发里叫“请求的生命周期”，理解它是看懂整个项目代码的钥匙。

1. 你在页面上选择起点 S2、终点 S6，点击「规划最短路径」。前端文件 frontend/src/views/MapView.vue 把这两个值收进一个 JavaScript 对象。
2. 前端调用 frontend/src/api.js 里封装的函数，真正发出一个 HTTP POST 请求，地址是 /api/path-planning/，请求体是 JSON 文本 {"start_node": ..., "end_node": ...}。
3. 请求先到达 Vite 开发服务器（端口 5173），Vite 的代理配置发现地址以 /api 开头，就把它转发给 Django（端口 8000）。这一步是为了绕开浏览器的跨域限制。
4. Django 根据 AGV/urls.py 里的路由表，把请求交给 SchedulingDecision 应用里对应的视图函数。

5. 视图函数做三件事：校验请求参数是否合法；调用 services.py 里的算法，用 Dijkstra 在图上搜索最短路径；把结果整理成字典。
6. 序列化器（serializers.py）把 Python 字典转换成 JSON。
7. JSON 沿着原路返回浏览器，Vue 把返回的节点序列画成地图上的红线。

关键认识：前端代码里没有任何“算法”，所有计算都在后端的 services.py 里。这种写法的好处是算法只写一遍，网页、手机 App、甚至别的程序调用同一套接口都能用。

1.5 项目目录导航

第一次打开这个项目，很容易被一堆文件夹吓到。其实只需要记住下面这几个位置。

路径	里面放什么	你需要看它的场景
AGV/settings.py	Django 全局配置：数据库连接、跨域、接口默认权限、分页大小	改数据库密码、理解接口为什么默认免登录
AGV/urls.py	总路由表，把 /api/、/api/vision/ 等前缀分给不同应用	想找一个接口对应哪段代码
SchedulingDecision/	调度内核：地图、AGV、任务的数据模型 + 全部调度与路径算法	任务 1、2、3 的核心代码
ManufacturingVision/	视觉模块：裂纹检测与工件识别算法、数据集管理、批量评估命令	任务 4、5
EquipmentMonitoring/	设备监测模块：合成训练数据、随机森林模型、规则告警	任务 6
DemandForecast/	订单预测模块：三种时间序列模型与自动选优逻辑	任务 7
frontend/src/views/	8 个页面的 Vue 组件，一个文件对应一个菜单	想改界面或找按钮背后的请求
frontend/src/api.js	把 8 个页面的所有 HTTP 请求封装在一个文件里	想查某个操作调用了哪个接口
datasets/vision/	视觉数据集：MT 裂纹数据集 + 合成工件数据集	任务 4、5 的输入数据
SchedulingDecision/management/commands/seed_demo.py	一键初始化演示数据的脚本	演示前恢复数据
run_project.py	同时启动 Django 和 Vite 的启动脚本	本地一键跑起来

记忆方法 五个应用的名字就是任务分组：SchedulingDecision 管任务 1/2/3，ManufacturingVision 管 4/5，EquipmentMonitoring 管 6，DemandForecast 管 7。看目录名

就知道该去哪个文件夹找代码。

第 2 章　把基础补回来

这一章是给“课学过但忘了”的人准备的。不用一次读完，遇到看不懂的名词回来查即可。

2.1 前端、后端、数据库，各干什么

- **前端（frontend/）**：在浏览器里运行的程序，负责把数据显示成表格、地图、曲线，并把用户的输入收集起来。本项目用的是 Vue 3。它自己不保存数据，一刷新就没了。
- **后端（Django 部分）**：在服务器（这里就是你自己的电脑）上运行的程序，负责业务逻辑和计算。页面上的每一个数字、每一条路线，都是它算出来发过去的。
- **数据库（MySQL）**：专门用来长期保存数据的软件。地图节点、AGV、任务、检测记录都存在这里。关掉程序再打开，数据还在，就是因为它们在数据库里。

三者的关系可以类比成餐厅：前端是服务员（传菜、听你点单），后端是厨房（真正做菜），数据库是仓库（长期存食材）。服务员不会做菜，厨房也不会把菜长期放身上。

2.2 API、REST、JSON 到底是什么

API（应用程序接口）就是前后端之间的“约定好的对话方式”。前端说“我要规划从 S2 到 S6 的路径”，地址是 /api/path-planning/，后端就必须按这个地址接话。这个地址和它的参数格式，就叫接口契约。

REST 是一种设计接口的风格，核心思想是“用地址表示资源，用动作表示操作”。本项目里 /api/tasks/ 代表“任务集合”，对它发 GET 就是查、POST 就是新增；/api/tasks/5/ 代表 5 号任务，对它发 PUT 是改、DELETE 是删。记住这四个动作，整个项目的接口你就能读懂一大半。

JSON 是前后端传数据的文本格式，长得像 Python 的字典和列表：大括号包的是字典（键值对），中括号包的是列表。它的好处是任何语言都能读。例如一条任务长这样：

```
{
  "task_no": "TASK-DEMO-001",
  "cargo_name": "电子元件箱",
  "cargo_weight": 35.0,
  "priority": 3,
  "status": "pending"
}
```

2.3 什么是 ORM，为什么不写 SQL

ORM 的全称是对象关系映射，说白了就是“用 Python 类来代表数据库表，用 Python 对象来代表表里的一行”。本项目的地图节点表在 models.py 里是这样定义的：

```
class MapNode(models.Model):
```

```
    code = models.CharField(max_length=32, unique=True)      # 节点编号，如 S1
    name = models.CharField(max_length=64)                    # 节点名称
    node_type = models.CharField(max_length=20, ...)          # 节点类型：取货点/放货点/路口…
    x = models.FloatField()                                   # 横坐标
    y = models.FloatField()                                   # 纵坐标
```

这段代码一执行，数据库里就会出现一张表；每一个字段就是一个列。之后想查所有节点，写 `MapNode.objects.all()` 就够了，不需要写 `SELECT * FROM ...`。Django 会把它翻译成 SQL。这就是为什么你能在不懂 SQL 的情况下做完整个项目。

一个必须知道的细节：改完 `models.py` 必须执行两条命令才能真正改到数据库——`python manage.py makemigrations`（生成改动记录）和 `python manage.py migrate`（把改动应用到数据库）。前者只是“写计划”，后者才是“动手术”。

2.4 Python 里你一定会反复看到的写法

写法	含义	本项目的例子
`class 类名:`	定义一类事物的模板	`class GraphRouter:` 定义一个带缓存的最短路径计算器
`self`	代表“这个对象自己”	`self.cache` 表示每台路由器自己带一个缓存字典
`def 函数名(参数):`	定义函数（也叫方法）	`def path(self, start, end)` 求两点间最短路径
字典 `dict`	键值对集合，用 `keys()` 取值	`distances = {start: 0.0}` 记录每个节点当前已知的最短距离
列表 `list`	有序集合，用 `append()` 追加	`orders = {agv.id: []}` 每台 AGV 一个任务顺序列表
`heapq`	优先队列，每次弹出最小值	`queue = [(0.0, start)]` 是 Dijkstra 的核心数据结构
`None`	表示“没有值”	`assigned_agv = None` 表示任务还没分配车辆
异常 `raise`	主动报错，中断执行	起点终点不连通时抛 `SchedulingError`
`with ... as ...`	进入一个上下文，退出时自动清理	`with transaction.atomic():` 保证调度过程要么全成功要么全回滚

其中最值得花时间弄懂的是字典和优先队列。本项目所有算法都在和这两个东西打交道：字典用来存“每个节点当前的最短距离”“每台车当前的任务列表”，优先队列用来每次取出“当前离起点最近的那个节点”。

2.5 运行环境里必须知道的几件事

- **`requirements.txt`**：列出项目依赖的第三方库及版本。换一台电脑部署，先执行 `pip install -r requirements.txt` 把库装齐。
- **虚拟环境**：一个独立的 Python 环境，避免不同项目的库版本互相干扰。本项目锁定 `numpy==1.26.4`，装进别的环境可能把对方的库降级，所以要单独建环境。

- **MySQL**：数据库服务，默认端口 3306。项目用的库名是 agv。数据库没启动，后端一启动就会报连接错误。
- **Node.js 与 npm**：前端 Vue 的运行环境。没有它，frontend 目录里的 npm run dev 无法执行。

```
# 后端启动（在项目根目录）
python manage.py migrate          # 建表
python manage.py seed_demo --reset  # 初始化演示数据
python manage.py runserver 127.0.0.1:8000   # 启动后端

# 前端启动（另开一个终端，进入 frontend 目录）
npm install
npm run dev                       # 启动前端，浏览器访问 http://127.0.0.1:5173
```

偷懒办法　项目根目录的 run_project.py 会一次性把上面两个服务都拉起来，直接 python run_project.py 即可。

第 3 章　任务 3：地图建模与最短路径规划

七个任务里，路径规划是最底层的一个。后面单台调度、多台调度算“这台车从这跑到那要多久”，问的都是路径规划。所以这一章值得先弄懂。

3.1 地图是怎么变成“图”的

仓库平面图在代码里被抽象成数据结构里的“图”（Graph）。做法非常直接：

- **节点（Node）**：地图上可以停车的每一个位置。本项目把仓库划成 12×8 的网格，扣掉 20 个被障碍占掉的格子，剩下 76 个节点。
- **边（Edge）**：两个相邻可通行节点之间的道路。相邻两格之间建立一条双向边，最终共 103 条。单格边长 4 米。
- **障碍（Obstacle）**：不可通行的矩形区域，共 10 个。它们不直接参与计算，而是通过“被扣掉的格子”影响图的结构—没有节点，自然就不会有路穿过去。

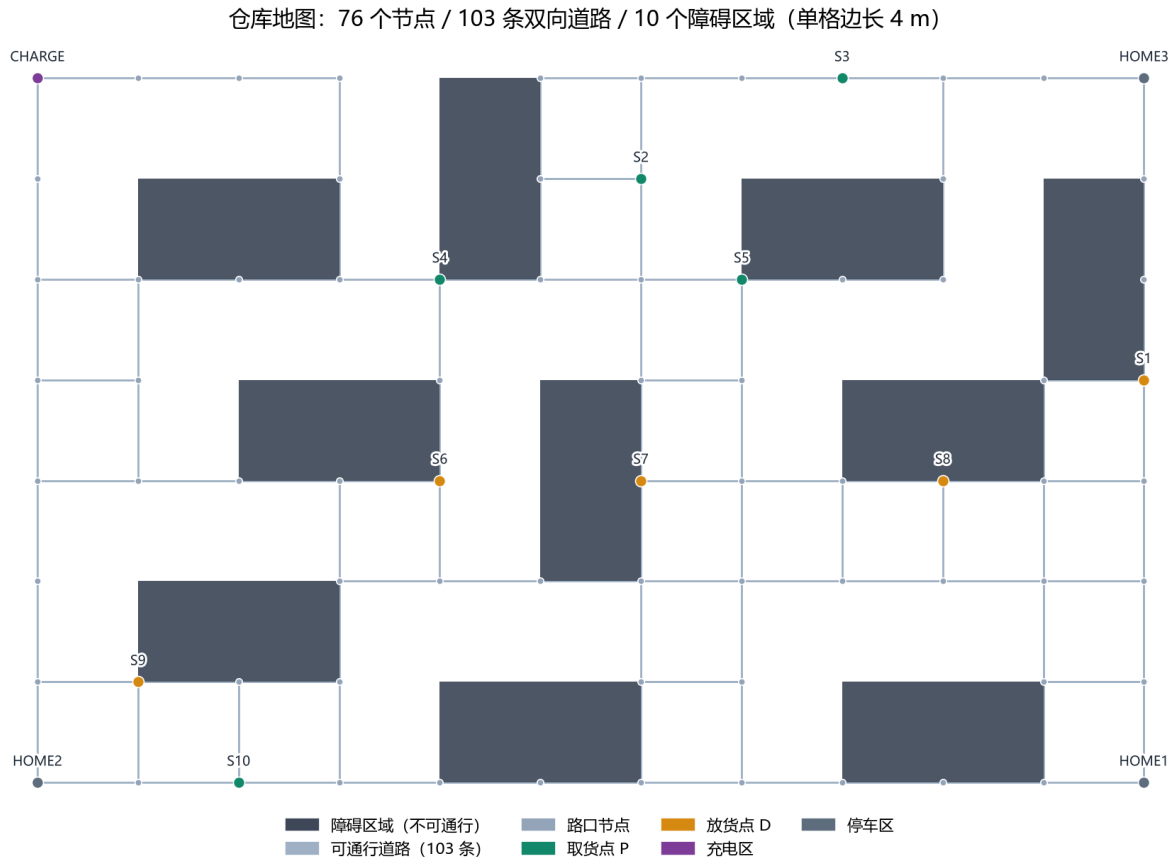


图 2　仓库地图的真实结构：深色方块是障碍区域，浅灰线是 103 条可通行道路，绿色圆点是取货点 P，橙色圆点是放货点 D

图里那些 S1 到 S10 就是任务书里说的“物料出发区”“加工区”“存放地”。灰色小点是普通路口，没有业务含义，纯粹是为了让路网连通。左上角 CHARGE 是充电区，另外三个 HOME 是停车区。

3.2 直觉：什么是最短路径

把图想象成一张地铁线路图，节点是车站，边是站与站之间的线路，边的权重就是距离（米）。最短路径问题就是：从起点站到终点站，怎么换乘总里程最短。

注意区分两件事：换乘次数最少 ≠ 距离最短。地铁里多坐一站但少换乘两次，往往总距离更短。这就是为什么不能用“广度优先搜索（BFS）”，而要用考虑权重的 Dijkstra 算法。

3.3 Dijkstra 算法：一句话说透

Dijkstra 的核心思想只有一句：从起点开始，每次挑出“当前已知距离最小”的那个节点，用它去更新它所有邻居的已知距离；已经被挑出来处理过的节点，后面不可能再变得更短，于是标记为“已确定”。

用一个小图手动推一遍。图中有 5 个节点，边和距离是：A-B=4、B-C=4、A-C=9、B-D=6、C-D=4、D-E=4。求 A 到 E 的最短路径。

轮次	挑出的节点	当前距离	用它更新的邻居	更新后的距离表
初始	—	—	—	A=0，其余全部为 ∞（无穷大，表示暂时不可达）
第 1 轮	A	0	B、C	B=4（经 A），C=9（直接 A-C）
第 2 轮	B	4	A、C、D	C 由 9 降为 8（A→B→C 更短！），D=10（经 B）
第 3 轮	C	8	B、D	D 尝试更新为 8+4=12，比已有的 10 大，放弃
第 4 轮	D	10	B、C、E	E=14（经 D）
第 5 轮	E	14	—	已到达终点，搜索结束

这个例子里有两个关键现象，答辩时如果被问到“为什么不能贪心”，就拿它举例：

- 第 2 轮把 C 的距离从 9 改成了 8——说明“先发现的路径不一定更短”，必须允许更新。
- 第 3 轮 D 已经有 10 了，用 C 走的 12 更差，所以不动——这就是 Dijkstra 能成立的原因：每次取出的节点距离已经是最优的。

3.4 换个角度看：最短距离场

另一种更有用的理解方式是把结果看成一张“距离场”。从起点开始向四周扩散，每个节点上都标着“从起点走到这里最少要走多少米”。算完距离场之后，从终点沿着“距离不断变小”的方向往回走，走出来就是最短路径。

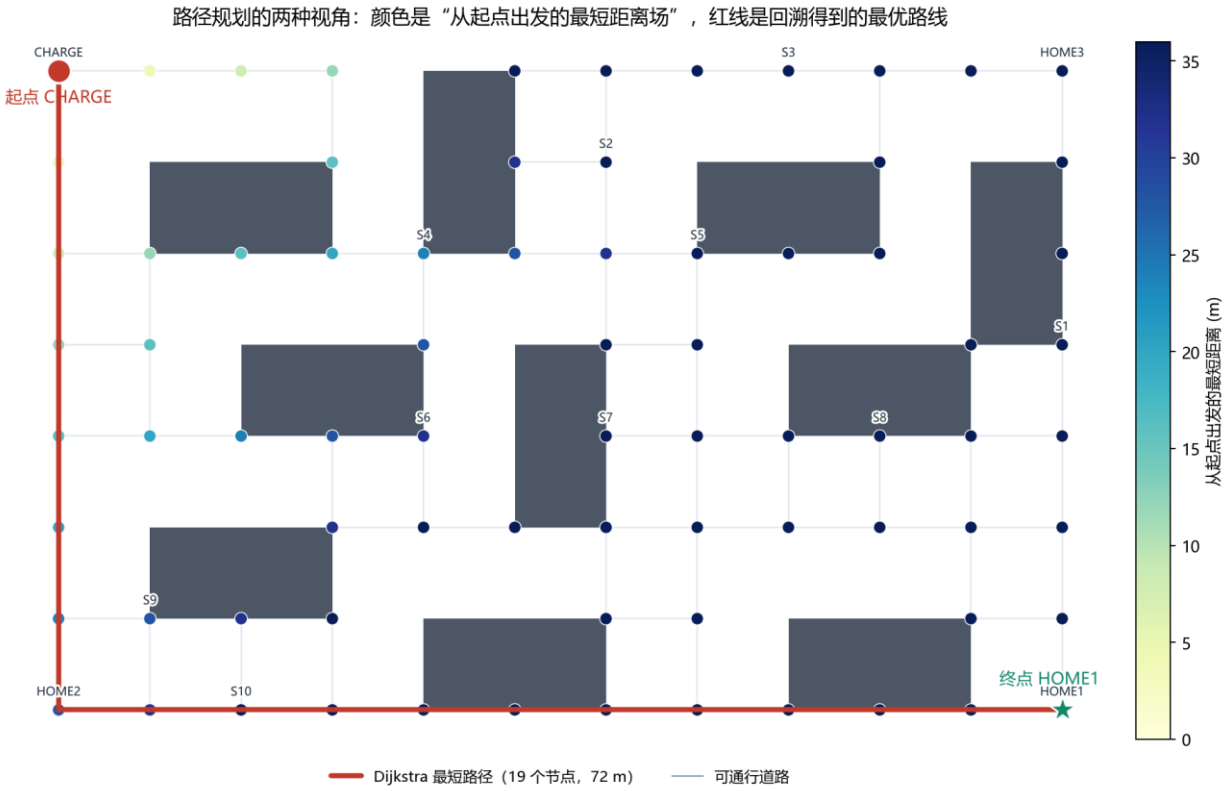


图 3　同一张地图的两种视角：节点颜色代表从起点出发的最短距离，红线是从终点回溯出来的最优路径（19 个节点，72 米）

这张图解释了为什么路径能自动绕开障碍。障碍区域里没有节点，距离场就只能从旁边绕过去。你不需要专门写“绕障代码”，只要求解算法在图上跑，绕障是图结构的自然结果。

3.5 代码在哪里，怎么读

```
# SchedulingDecision/services.py

class GraphRouter:
    """带最短路径缓存的 Dijkstra 路由器。"""
    def __init__(self, graph):
        self.graph = graph        # 邻接表：{节点 id: [(邻居 id, 距离, 限速), ...]}
        self.cache = {}           # 缓存：算过一次的 (起点,终点) 直接复用

    def path(self, start, end):
        key = (start, end)
        if key not in self.cache:
            self.cache[key] = self._dijkstra(start, end)
        return self.cache[key]
```

读这段代码要抓住三个点：

8. `graph` 是邻接表。它用一个字典表示整张图：键是节点编号，值是这个节点所有邻居的列表。用邻接表而不是邻接矩阵，是因为地图很稀疏（76 个节点只有 103 条边），存矩阵会浪费大量空间。

9. heapq 是优先队列。算法需要不断取出“当前距离最小的节点”，用优先队列实现这个动作只需要 O(log n) 时间，比每次遍历一遍所有节点快得多。
10. cache 是缓存。一次调度里同一对起点终点会被反复查询，缓存让第二次查询变成常数时间。这也是本项目能让变邻域搜索跑几千次迭代还不算慢的关键原因之一。

3.6 路径之外的约束检查

找到最短路不等于可以直接执行。项目里还对结果做了两层校验：

- **电量约束**：路径长度换算成预计耗电，如果指定了执行车辆而它的剩余电量低于安全阈值，接口会拒绝这次规划，而不是硬算出来给一个跑不完的方案。
- **道路拓扑约束**：起点或终点如果不在图里、或者两点之间没有连通路径，算法会抛出异常，接口返回明确的错误信息，而不是返回一条错误的路线。

换算关系：行驶时间 = 路径总长 ÷ 车速；预计耗电 = 路径总长 × 单位里程耗电率。本项目里 AGV 车速取 1.2 m/s，所以在图 2 的例子里，72 米的路线大约需要 60 秒。

3.7 关于任务 3 的常见提问

可能的提问	回答要点
为什么用 Dijkstra 不用 A*？	地图规模很小（76 个节点），两种算法在速度上没有可感知区别；而且调度算法需要在“任意两点之间”反复查询距离，事先算全源最短路缓存起来更划算。A* 需要为每个查询单独设计启发函数，收益不明显。
为什么不用 BFS？	BFS 只适用于边权相等的情况。本项目每条边的距离虽然都接近 4 米，但存在跨格的长边，边权不相等，用 BFS 会得到“步数最少”而不是“距离最短”的路线。
地图为什么用网格？	工业现场的 AGV 跑在规则巷道里，网格建模简单、可控、易验证；同时障碍用矩形表示可以和实际货架、设备的位置一一对应。
能算出多条备选路径吗？	当前只返回最优的一条。如果要备选，可以把最优路径上的某条边临时删除再求一次，这叫“K 短路”的简化做法，是后续可以扩展的点。

第 4 章　任务 1：单台 AGV 的多任务顺序优化

4.1 问题到底难在哪

假设仓库里有 1 台空闲 AGV，同时有 4 个搬运任务在排队。每个任务都要“先跑到取货点，再跑到放货点”。这台车做完 4 个任务之后，总行驶里程取决于它按什么顺序做。

顺序有多少种？4 个任务有 4×3×2×1 = 24 种；6 个任务有 720 种；10 个任务有 362 万种；15 个任务有 1300 多亿种。这个增长速度叫“阶乘级增长”，是最典型的组合爆炸。

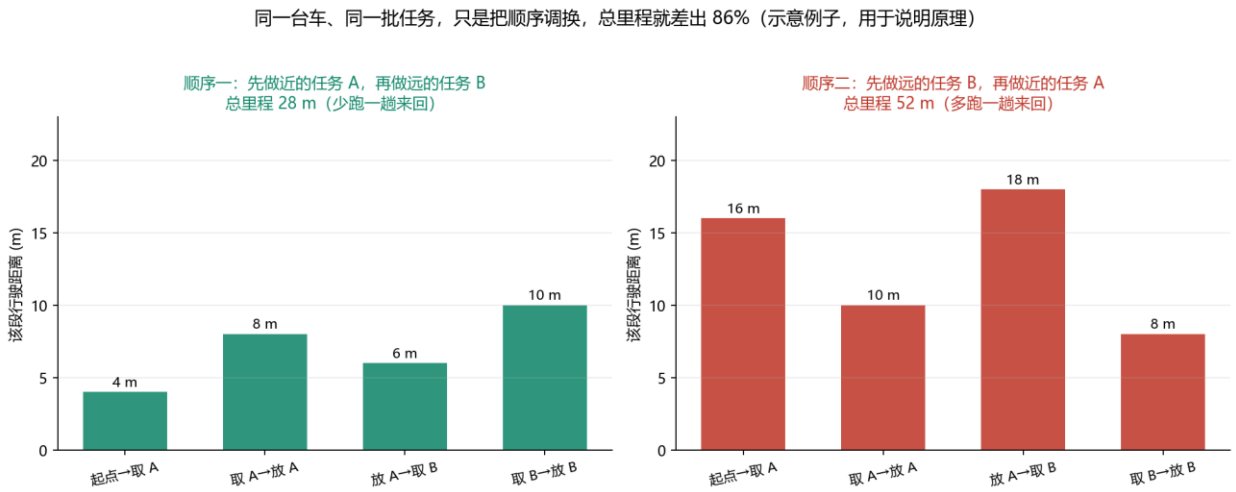


图 4　示意例子：同一台车、同一批任务，先做近的还是先做远的，总里程差了 86%（本图为说明原理构造的小例子，非真实数据）

图里第二个顺序之所以差得多，是因为先接了远端的任务 B，回来再做近处的 A，等于把长距离路段来回跑了两趟。真实仓库里任务分散时，顺序带来的差异经常在 20% 到 40% 之间。所以任务 1 的核心不是“能不能做完”，而是“怎么排序最省”。

4.2 从穷举到动态规划

先说结论：本项目对 10 个任务以内的情况，用状态压缩动态规划求出的顺序是“数学上最优的”，不是近似。

动态规划的核心思想是“不重复解决同一个子问题”。在上面的问题里，有两个信息决定了当前的处境：① 现在已经完成了哪些任务；② 上一个完成的是哪个任务（因为车停在它的放货点）。只要这两个信息相同，后面的最优做法就完全一样，跟前面是按什么顺序走到这一步的无关。

用位掩码表示“完成了哪些任务”。比如有 5 个任务，用一个二进制数 mask 表示完成情况：mask = 00101（十进制 5）表示第 1 个和第 3 个任务已完成。这样一套状态就是 (mask, last)，其中 last 是最后完成的任务编号。

```
dp[mask][last] = 完成集合 mask 中的所有任务、且最后一个做的是 last 任务时，所需的最小总里程

转移方程：
dp[mask | (1<<k)][k] = min( dp[mask][last] + 转移里程(last -> k) )   对每个未完成的任务 k

其中 转移里程(last -> k)
   = 距离( last 的放货点 -> k 的取货点 )   # 空驶
   + 距离( k 的取货点   -> k 的放货点 )    # 载货行驶
```

最后答案取所有 dp[全部完成][某个 last] 的最小值，再沿着记录下来的“前一个状态”回溯，就得到任务顺序。时间复杂度是 $O(n^2 \cdot 2^n)$：n=10 时大约 10 万次运算，毫秒级完成；而穷举需要 362 万次。

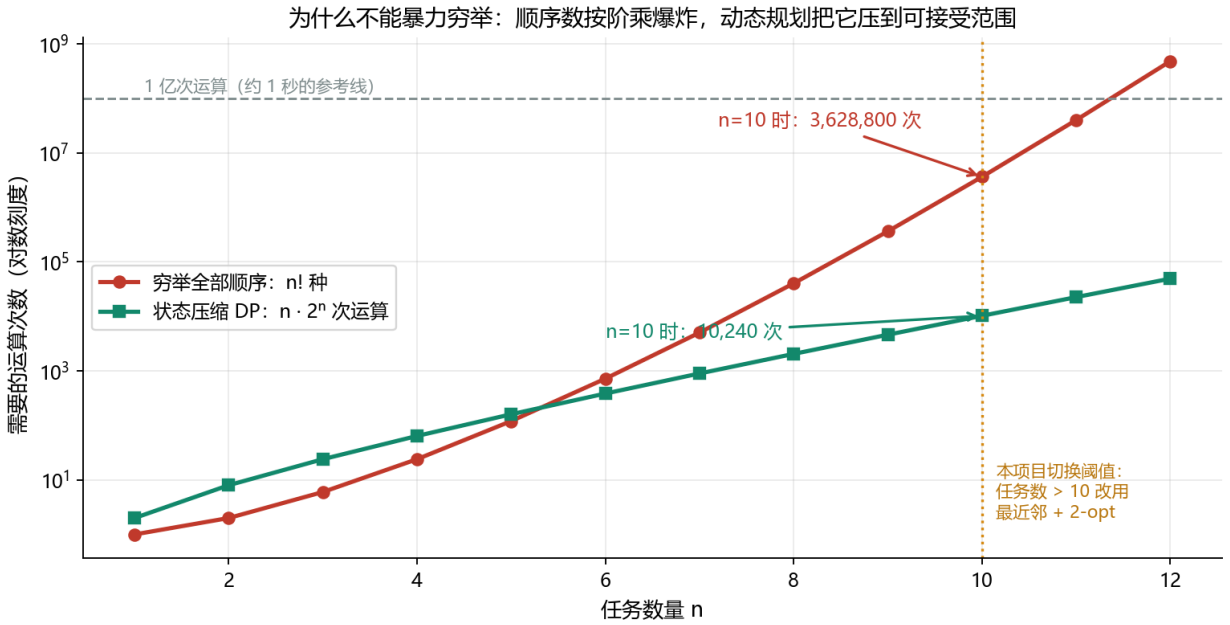


图 5　运算量对比：穷举顺序数是 n!（阶乘级），状态压缩动态规划是 $n \cdot 2^n$（指数级但底数小得多），n=10 时差了两个数量级

答辩要点　如果被问“动态规划为什么正确”，回答两句话：一是无后效性—当前状态只由（已完成集合，最后任务）决定，与到达方式无关；二是最优子结构—全局最优解一定由子问题的最优解拼接而成。

4.3 任务数超过 10 怎么办

n 再大下去，2^n 也会爆炸（n=20 时是 100 万倍的状态数）。项目设定了阈值：超过 10 个任务改用启发式方法，牺牲“保证最优”换取“能在可接受时间内得到足够好的解”。

11. 最近邻（Nearest Neighbor）：从当前位置出发，每次选择“最近的那个未完成任务”。构造快，但容易走进局部最优。

`12.2-opt` 优化：在得到的顺序里任取两段，把中间那一段倒过来，如果总里程变短就保留。反复尝试所有可能的翻转，直到不能再改进。

`2-opt` 为什么有用：假设当前顺序是 `A→B→C→D`，如果你发现 `A→C→B→D` 更短，那么把**“B→C”**这一小段反过来就得到了新顺序。它每次只动一小段，代价低但改进了不少。

4.4 用哪台车？——综合成本选车

如果用户没有指定车辆，系统要自己挑一台。挑车的逻辑不是**“随便找一台空闲的”**，而是给每台空闲车算一个综合成本，取最小：

考虑因素	含义	影响方向
空驶距离	车当前位置到第一个取货点的距离	越大越差
剩余电量	评价这台车能不能撑完全程	越低越差，低于安全阈值直接淘汰
载重裕量	车辆额定载重减去所载货物重量	越小越差，装不下直接淘汰
车辆状态	空闲才有资格参与分配	非空闲不参与

这套评分的好处是它不仅考虑**“现在谁最近”**，还考虑**“这台车做完之后还有没有电做下一批任务”**，避免出现一台车被反复派活、其它车闲着的情况。

4.5 运行结果长什么样

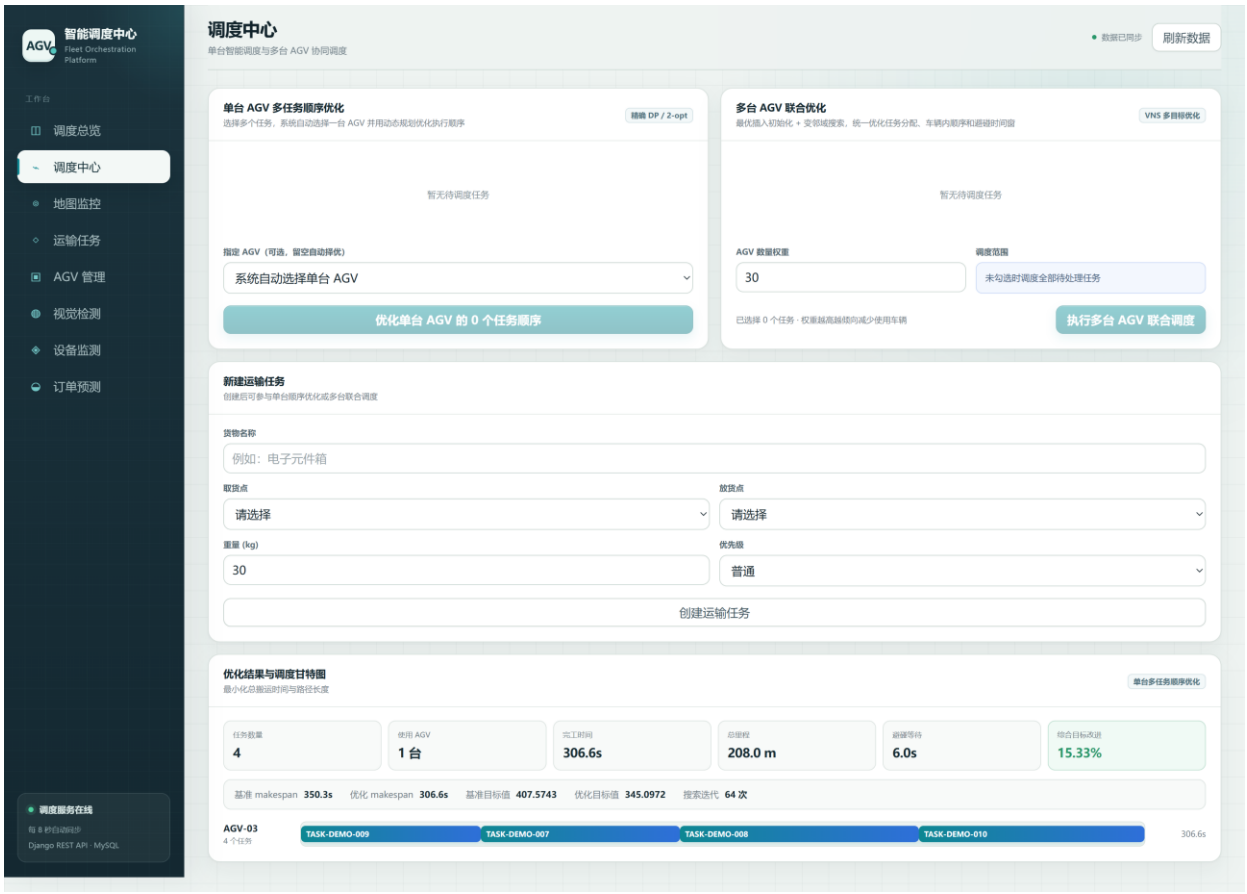


图 6　单台 AGV 多任务顺序优化的真实运行结果：4 个任务全部交给 AGV-03，优化后的顺序是 TASK-DEMO-009 → 007 → 008 → 010

看这张图的时候注意右下角那一行数字，它们就是这一章所有概念的落地：

- **任务数量 4、使用 AGV 1 台**：符合任务 1 的设定，一台车连续做多个任务。
- **完工时间 306.6 s**：整批任务从开始到结束的总耗时，也就是常说的 makespan（最大完工时间）。
- **总里程 208.0 m、避碰等待 6.0 s**：算法同时统计了行驶距离和因为避免冲突而额外增加的等待时间。
- **基准 makespan 350.3 s → 优化 makespan 306.6 s，综合目标改进 15.33%**：这是最值得在答辩里讲的一组数字。基准算法的含义是“按用户提交的顺序（按优先级排序）执行”，优化算法重新排了任务顺序，把完工时间缩短了 15.33%。
- **搜索迭代 64 次**：说明优化过程做了 64 轮状态扩展或邻域尝试。

甘特图怎么看：横轴是时间，蓝色条是从取货点到放货点的载货行驶段，条与条之间的空隙是空驶段和装卸等待。四个任务首尾相接，中间没有闲置，说明顺序排得比较紧凑。

4.6 关于任务 1 的常见提问

可能的提问	回答要点
为什么单台调度要用动态规划，而不是贪心？	贪心的每一步都只看眼前最近的任务，容易在最后留下一个很远的任务。动态规划枚举了“已完成集合 × 最后任务”的全部状态，所以在 n≤10 时能保证全局最优。
为什么阈值定在 10？	n=10 时状态数约 1 万，n=11 翻倍到 2 万，n=15 就到 50 万级别；超过 10 之后启发式的性价比更高。这个阈值是精度和速度的折中，可以按机器性能调整。
2-opt 能保证最优吗？	不能，它是局部搜索，只能保证比初始解好。要更强的效果可以叠加 3-opt 或 Or-opt，代价是计算时间上升。
基线是怎么算的？	基线是“按优先级和提交时间排序后顺序执行”，也就是不做任何优化的自然做法。改进率 =（基准目标值 − 优化目标值）÷ 基准目标值。

第 5 章　任务 2：多台 AGV 联合调度

5.1 问题从“排序”变成了“分配 + 排序”

单台车的问题只有一个决策：任务按什么顺序做。到了多台车，决策变成了两个，而且这两个决策互相影响：

13. 每个任务分配给哪台 AGV（分配决策）
14. 每台 AGV 内部按什么顺序执行分到的任务（排序决策）

这两件事必须一起决定。因为如果你先按某个规则分好任务，再各自排序，很可能分得就不合理；反过来先排序再分配也一样。这类问题在运筹学里属于 NP 难问题（VRP，车辆路径问题），任务数量大时不存在快速求出精确最优解的算法，所以工程上的做法都是“找一个足够好的解”，本项目也是如此。

5.2 基准算法：一个讲得清的“笨办法”

为了衡量优化算法到底有没有用，必须先有一个对照标准。项目里的基准算法非常直白：

按任务优先级从高到低、同级按提交时间排序，对每个任务，遍历所有空闲 AGV，把它追加到某台车的任务列表末尾，算一下这台车完工时间是多少，挑最早完成的那台。

这个算法是贪心的—每一步都选“眼前最优”，不考虑后面会怎样。它的优点是快、容易理解、结果稳定；缺点是任务一多就容易出现“所有活都压到同一辆车身上”的情况。优化算法的改进率，就是相对这个基准算出来的。

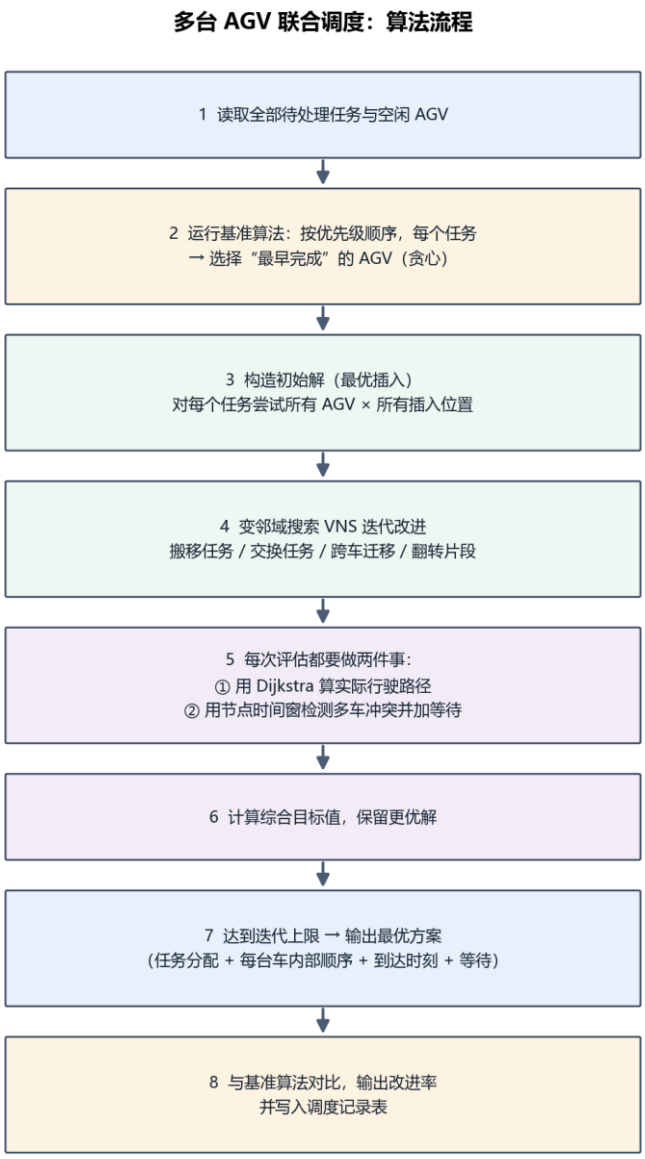


图 7　多台 AGV 联合调度的完整流程：先算基准，再构造初始解，然后反复邻域搜索改进，最后与基准对比输出改进率

5.3 初始解：最优插入

变邻域搜索需要一个起点，也就是一个“可行的任务分配方案”。项目用最优插入法构造这个起点：

15. 任务按优先级排序，一个一个地放。
16. 放每个任务时，尝试所有空闲 AGV 的 × 所有可能的插入位置。
17. 对每一个候选方案，立刻用路径规划算实际里程和完工时间，算出目标值。
18. 保留目标值最小的那个方案，处理下一个任务。

“最优插入”比基准算法的“追加到末尾”要强，因为它允许把一个任务插到某个任务列表的中间，也就是允许“顺路捎带”—这在有多个任务时非常关键。

5.4 变邻域搜索（VNS）在做什么

邻域搜索的思想：从一个方案出发，对它做一点小改动，得到一个新方案；如果新方案更好就接受，再继续改；如果变差了就换个改法。反复做，方案就会越改越好。

变邻域的意思是：不只用一种改法，而是准备了几种不同的“改法”（邻域），一种改不动了就换另一种。本项目用到的改动包括：

邻域动作	具体做什么	想解决的问题
搬移（Relocate）	把一个任务从它的位置拿出来，插到同一个任务列表的另一个位置	调整单车内部的顺序
交换（Swap）	把两个任务的执行顺序对调	快速修正明显的顺序错误
跨车迁移（Cross-move）	把一个任务从 AGV-1 的列表挪到 AGV-2 的列表	平衡各车负载
片段翻转（2-opt）	把任务列表里的一段整体倒序	消除交叉路线

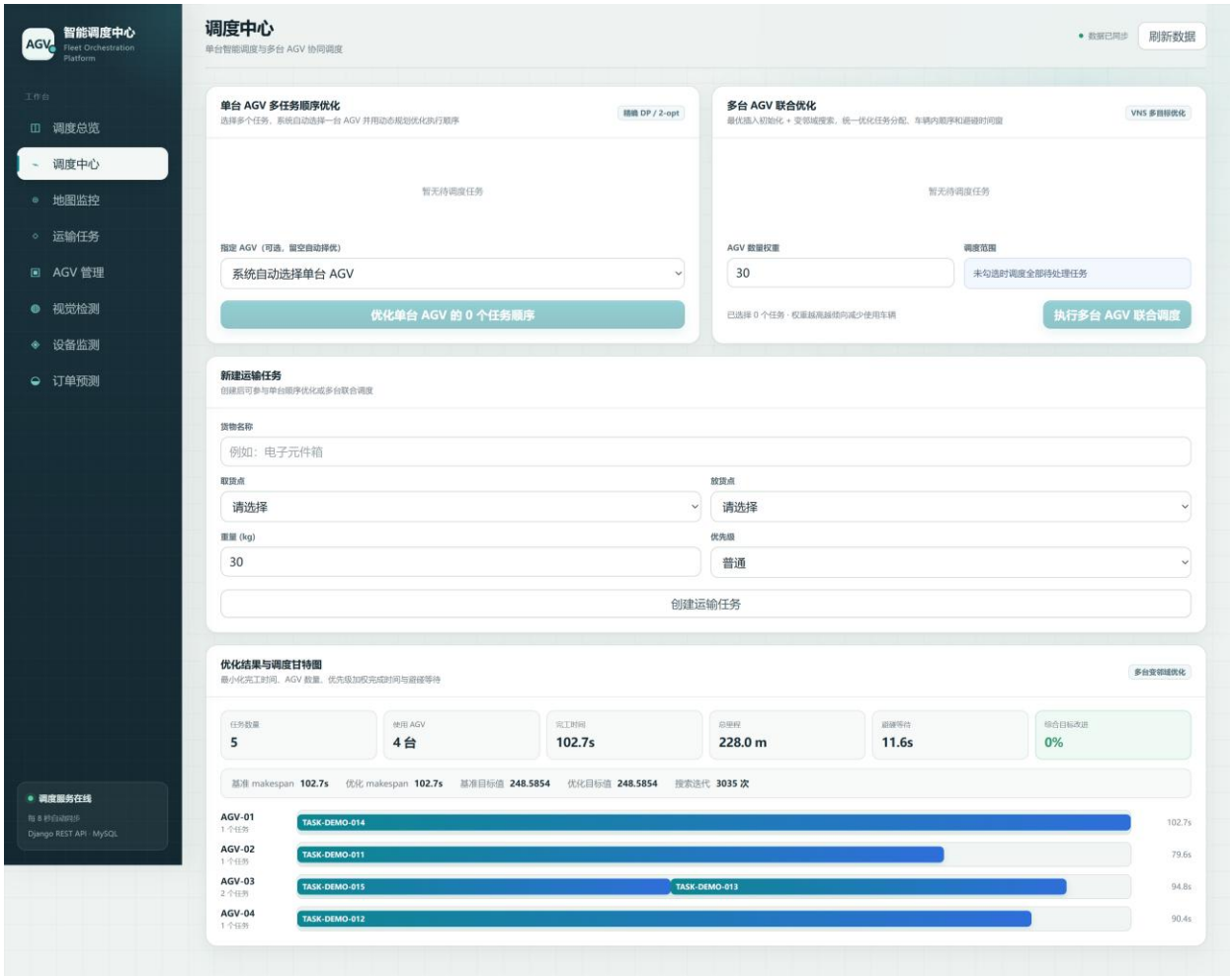


图 8　多台 AGV 联合调度的真实运行结果：5 个任务分给 4 台车，AGV-03 连续执行 2 个任务，其余各 1 个

这张截图里的甘特图是这一章最值得讲清楚的东西：

- **AGV-01 与 AGV-04 各只有 1 个任务**：任务分配时这两台车被派到就近的任务上。
- **AGV-03 有 2 个任务且首尾相接**：说明算法判断让同一台车连着做更划算，而不是启用第 5 台车（本项目一共 4 台）。
- **避碰等待 11.6 s**：这部分时间不是行驶，而是为了不和别的车撞上而被迫等待，下一节专门讲。
- **搜索迭代 3035 次**：变邻域搜索在 3035 次邻域尝试后收敛。这个数字越大说明搜索空间越大，也说明算法确实在认真找。

5.5 多车避碰：节点时间窗预约

多台车同时跑，就会出现“两台车想在同一时刻经过同一个路口”的情况。这个问题在单台车调度里不存在，是多车调度特有的。项目用一个叫“节点时间窗预约”的机制来解决，思路和会议室预订很像：

19. 每台车在规划路线时，会算出自己“预计到达每个节点的时刻”。
20. 把这个时段（到达时刻 ± 安全间隔）登记在这个节点上。
21. 如果发现某个节点已经被别的车登记了、而且时间上重叠，就在进入该节点之前插入一段等待时间，把到达时刻推迟到冲突结束之后。
22. 等待会顺延该车后续所有节点的到达时刻，所以这个检查要在路线生成的过程中按顺序做。

安全间隔是一段预留时间裕量，避免两台车刚好前后脚进入同一路口。本项目把等待时间计入总目标，所以算法不会无限制地插等待——等待多了目标值变差，它就会去找别的分配方案。

> **一句话总结** 时间窗预约让“冲突”从一个需要特殊处理的事故，变成目标函数里的一个普通成本项（等待秒数）。这体现了建模的一个常用思路：把麻烦事量化成代价，交给优化算法自己权衡。

5.6 综合目标函数：算法到底在“最小化”什么

单台调度只需要最小化总时间，多台调度要同时权衡好几件事。项目把多目标合成一个加权和：

```
目标值 objective =
      makespan                          # 全部任务完成所需的总时间
    + agv_weight    × 使用 AGV 的数量     # 用的车越少越好（权重可调）
    + wait_weight   × 避碰等待总秒数       # 等待越少越好
    + distance_weight × 总行驶里程        # 里程越短越好
    + priority_weight × 优先级加权完成时间   # 高优先级任务尽量早完成
```

权重决定了算法的偏好。界面上那个「AGV 数量权重」输入框就是 agv_weight：把它调大，算法会更倾向于“少用车、多绕路”；调小，算法会更在意总时间。这种“把多个目标加权求和”的做法叫加权和法，是最常用的多目标处理方法，优点是简单、可解释；缺点是需要人为调权重。

5.7 一个必须提前知道的坑：改进率可能显示为 0

项目刚做完时，数据库里仅有的两条调度记录，综合目标改进率都是 0.00%。这不是 bug，而是演示用例选得不好：当时是 4 个任务配 4 台空闲 AGV，恰好一台车做一个任务，基准算法（贪心）和变邻域搜索都得到了同一个解，所以改进率自然是 0。

要演示出改进效果，必须构造“任务数多于可用车辆数”的场景，让基准算法和优化算法产生分歧。做法很简单：准备 6 到 8 个待处理任务，只启用 3 到 4 台 AGV，把 AGV 数量权重设成 30 左右，再点「执行多台 AGV 联合调度」。实测这样能稳定跑出正的改进率（参考：6 个任务配 4 台车，实测改进率 6.44%）。

另外，改进率为 0 本身也是可以讲的：它说明在这个规模下贪心算法已经足够好，优化的价值要在更大规模或更强约束下才体现。答辩时这样回答，比硬说“算法很厉害”要可信得多。

5.8 关于任务 2 的常见提问

可能的提问	回答要点
为什么不用遗传算法、蚁群算法？	变邻域搜索实现简单、参数少、结果可复现（固定迭代次数和随机种子），在本项目的规模下（几个到几十个任务）效果和更复杂的元启发式相当。选它主要是工程上的性价比考虑。
算法怎么保证不撞车？	不是靠算法“保证”，而是靠每次评估方案时都执行时间窗预约，把冲突转成等待时间计入目标值。不可行的方案在评估阶段就会被扣分。
改进率越高说明算法越好吗？	不一定。改进率取决于基准算法的强弱和问题规模。基准越弱、规模越大，改进率看起来越高。比较算法时要固定问题规模，并同时报告绝对指标（如完工时间）。
为什么权重可以人工调？	因为多个目标之间本来就没有唯一正确答案：是“少用一台车”重要，还是“早完成 10 秒”重要，这属于管理决策。把权重开放给使用者，是把这个决策权还给现场。
3 千多次迭代要跑多久？	在本项目的规模下是秒级。快的原因有两个：一是 Dijkstra 结果带缓存，二是每次邻域改动只重新评估受影响的那一段，不是全量重算。

第 6 章　任务 4 / 5：机器视觉

从这一章开始，问题从“怎么排得更好”变成“怎么从图像里看出信息”。好消息是：本项目用的是传统图像处理方法，不需要训练神经网络，原理可以用高中的数学直觉理解。

6.1 图像在计算机里到底是什么

一张彩色图片就是三个二维数组叠在一起，分别记录每个像素的红、绿、蓝亮度，取值在 `0` 到 `255` 之间。把它转成灰度图后，就只剩一个二维数组，每个位置一个数：越接近 `0` 越黑，越接近 `255` 越白。

于是所有图像处理都变成了数组运算。“去噪”就是让相邻像素的值更接近；“找出暗色细线”就是找那些“自己比左右两边都暗”的位置。本项目裂纹检测的所有步骤，都可以用这句话解释。

另外两个常见术语顺手记住：

- **滤波（`Filtering`）**：用一个小窗口滑过整张图，每个位置用窗口内像素的加权平均替换中心像素。高斯滤波的权重按距离远近呈钟形分布，越近的像素影响越大。
- **阈值分割（`Thresholding`）**：设定一个数，大于它的像素判为“目标”，小于它的判为“背景”。这一步把灰度图变成了黑白两值图。

6.2 任务 4 裂纹检测的完整处理链

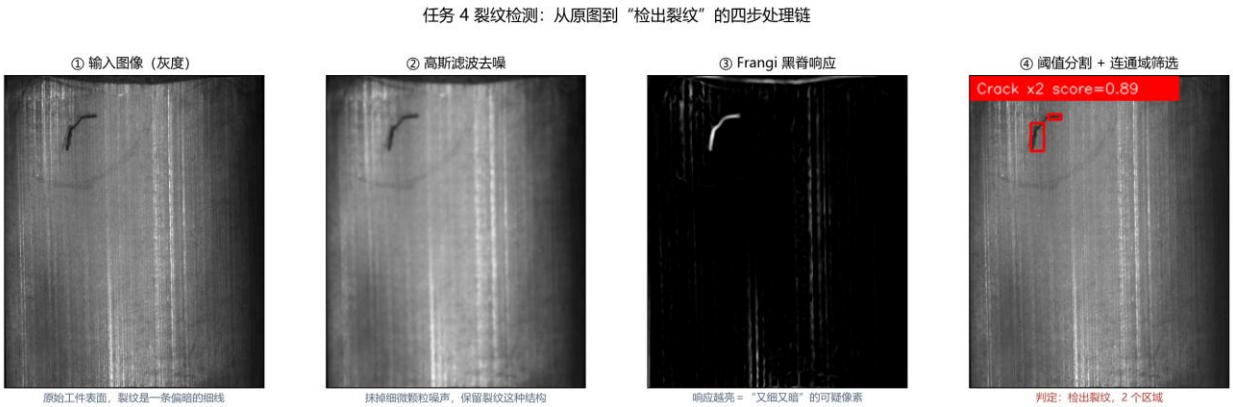


图 9　裂纹检测的四步处理链：从原始灰度图，经过滤波、黑脊响应、阈值分割与连通域筛选，最终判定“检出裂纹”

逐步解释这四步在做什么，以及为什么必须这样做：

步骤	做了什么	为什么需要这一步
① 转灰度并缩放	把彩色图转成灰度图，并把长边限制在 512 像素以内	算法只关心明暗，不需要颜色；统一尺寸保证后面的阈值参数在不同图片上都适用
② 高斯滤波	用一个 5×5 左右的高斯窗口平滑图像	工件表面的颗粒噪声会让每个像素都显得“可疑”，先抹掉细碎噪声，只留下成片的结构

步骤	做了什么	为什么需要这一步
③ Frangi 黑脊响应	在每个像素上算一个分数，分数越高说明该像素“又细又暗，而且两侧更亮”	这是全流程最关键的一步。它把“裂纹”这种线状暗结构的特征直接量化成一个分数，块状的污渍和台阶状的边缘得分都很低
④ 阈值 + 连通域筛选	给分数设阈值得到黑白图，再按面积、细长比、一致性筛选掉小块噪声	阈值分割会带出许多零星小点，靠连通域的形状特征把它们剔除，只保留真正的裂纹

6.3 关键知识点：什么是“黑脊”，Frangi 在算什么

这一节是任务 4 最容易讲不清、也最容易被追问的地方，值得多花点时间。

先说结论：Frangi 滤波器不“找边缘”，它找的是“像血管、裂纹那样又细又暗的线状结构”。它的做法是在每个像素周围计算二阶导数（曲率），然后看曲率在两个方向上有什么不同。

直觉解释：想象你站在一条裂纹上。沿着裂纹方向走，明暗几乎不变（平坦）；垂直于裂纹方向走，你会先下到裂纹底部再爬上来（像一个山谷）。这种“一个方向平坦、另一个方向是山谷”的形状，正是线状结构的数学特征。

实现上用 Hessian 矩阵描述这种局部曲率，它的两个特征值 λ_1、λ_2 分别代表两个方向上的弯曲程度：

特征值情况	局部形状	判定	本项目怎么处理
λ_1、λ_2 都很小	平坦区域	不是结构	得分 0
一个很小、一个很大且为正	亮背景上的暗线（山谷形状）	疑似裂纹	得分高—这是我们要找的
一个很小、一个很大且为负	暗背景上的亮线（山脊形状）	不是裂纹	得分 0
两个都很大	斑点或角落	不是裂纹	得分 0，被抑制

多尺度的含义：裂纹有粗有细。项目用了 1.0、1.4、2.0、2.8 四个尺度（本质上是四种窗口大小）分别计算一遍，然后取最大值。这样细裂纹和粗裂纹都不会漏掉。

为什么不用任务书里写的“边缘检测算子”：这个问题在 6.6 节单独回答，先记住这一句：边缘检测算子（如 Canny、Sobel）找的是“明暗变化剧烈的位置”，它会把裂纹的边缘和工件轮廓、刀痕、划痕的边缘一起找出来，无法区分“裂纹的边”和“工件外轮廓的边”；而黑脊响应带有“细”和“暗”这两个物理约束，更适合本任务。

6.4 怎么判断检测结果好不好

这是所有检测类任务都必须回答的问题。先用一张表把四个基础指标讲清楚。假设一共 100 张图，其中 40 张真的有裂纹：

指标	计算方式	通俗含义	本项目实测
准确率 Accuracy	(TP + TN) ÷ 总数	所有判断里，判对了的比例	0.8662

指标	计算方式	通俗含义	本项目实测
精确率 Precision	TP ÷ (TP + FP)	判为“有裂纹”的里面，真的有裂纹的比例	0.7903
召回率 Recall	TP ÷ (TP + FN)	真的有裂纹的里面，被找出来的比例	0.8596
F1 分数	2PR ÷ (P + R)	精确率和召回率的调和平均，两者的平衡	0.8235

其中 TP 是“有裂纹且判为有裂纹”，FP 是“没裂纹却判成有裂纹”（误报），FN 是“有裂纹却没查出来”（漏报），TN 是“没裂纹也判为没裂纹”。这四个词在答辩里出现的频率极高，一定要能对上号。

工业场景里要特别注意：漏报和误报的代价是不对等的。漏掉一条裂纹可能意味着一个有缺陷的零件被装到机器上，代价很高；而误报只是让人多复查一次。所以在实际产线上，通常宁愿牺牲一点精确率，也要把召回率保住。

除了“判对没判对”，还要看“画得准不准”。这就用到 IoU（交并比）：

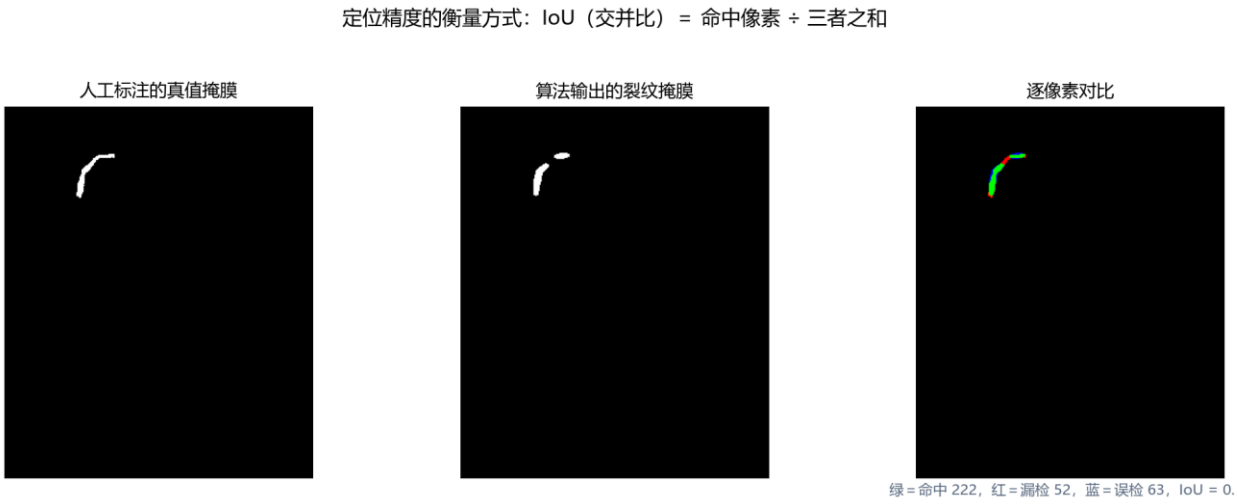


图 10　定位精度的衡量：绿色是命中像素，红色是漏检，蓝色是误检；IoU = 命中 ÷（命中 + 漏检 + 误检）

本项目实测平均 IoU 只有 0.14，这是这个模块最明显的短板。原因在于算法给出的裂纹区域偏细、偏保守，而人工标注的裂纹区域偏宽，两者重合部分不多。但它对“有没有裂纹”这个判断本身影响不大——F1 仍然是 0.82。回答时要主动区分“分类指标”和“定位指标”，不要含糊过去。

6.5 任务 5 工件识别：从像素到“这是什么形状”

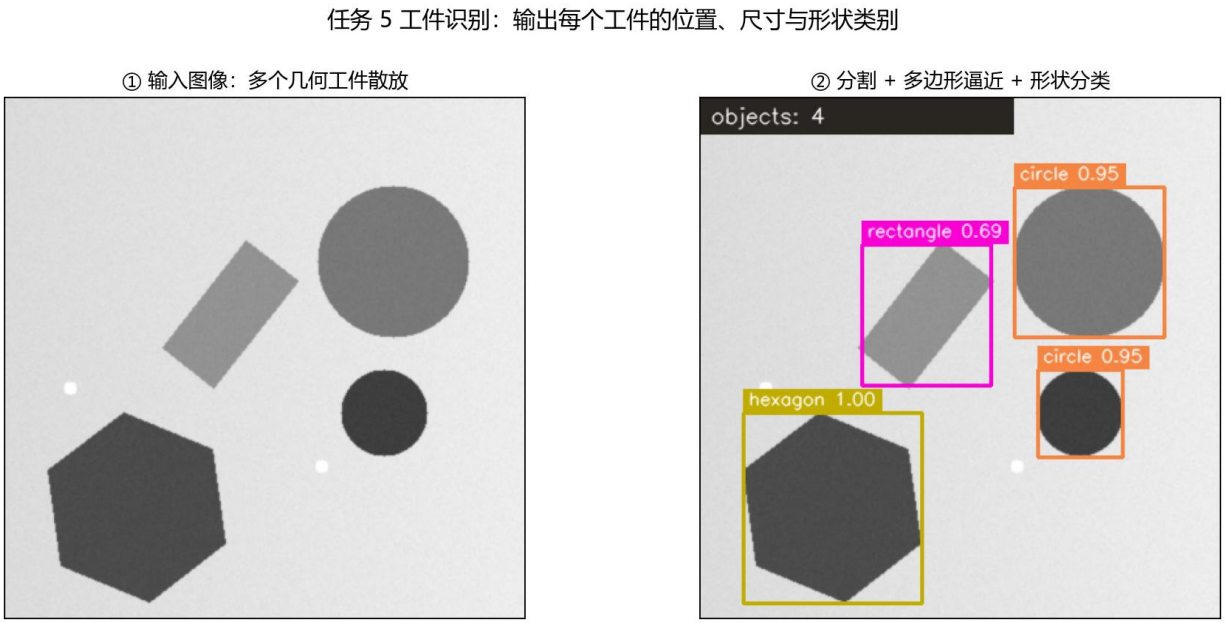


图 11　工件识别的真实运行结果：4 个工件被分别框出，标注了形状类别与置信度

这一模块的流程比裂纹检测更直观，都是可以用肉眼验证的步骤：

23. 双边滤波：既能去除噪声，又能保住工件的边界不被模糊掉（这是它比高斯滤波更适合这一步的原因）。
24. Otsu 阈值分割：自动找出一个把前景（工件）和背景分开的阈值。它的原理是让分出来的两堆像素内部尽量相似、彼此尽量不同，不需要人工指定阈值。
25. 形态学开闭运算：先腐蚀再膨胀（开运算）去掉小噪点，再闭运算填上工件内部的小孔洞。
26. 轮廓提取：把分割结果里每一块连通区域的边界描出来。
27. 多边形逼近：用最少的直线段去逼近轮廓。如果逼近结果只有 4 个顶点，它很可能是矩形或正方形；顶点很多则可能是圆形。
28. 形状分类：结合顶点数量、边长比例、圆度等特征，判定为圆形、三角形、正方形、矩形或六边形，并输出包围框与中心坐标。

为什么任务 5 需要输出中心坐标：因为任务书里写明这个信息是给机械臂抓取用的。机械臂要抓一个工件，至少要知道它的位置（中心点）和姿态（包围框的方向），所以识别算法的输出不能只有“这是什么形状”，必须有坐标。

6.6 与任务书描述的差异，以及实测指标

课程任务书对任务 4 的描述里提到了“应用边缘检测算子”和“对所选区域设定大小阈值”。本项目的实现和这句话有两处不同，这是答辩时几乎一定会被问到的地方，建议主动说明，而不是等老师来挑。

任务书表述	本项目做法	为什么这样做

任务书表述	本项目做法	为什么这样做
使用图像滤波及增强突出裂纹特征	高斯滤波 + 多尺度 Frangi 黑脊响应	完全符合。黑脊响应本身就是一种“特征增强”手段，而且针对性更强
应用边缘检测算子识别工件及裂纹边界	未使用 Canny/Sobel 等边缘算子，改用黑脊响应 + 连通域筛选	边缘算子无法区分裂纹边缘与工件轮廓、加工刀痕的边缘；黑脊响应用“细且暗”的物理约束把非裂纹结构直接压下去，实测 F1 更高
对所选区域设定大小阈值判断裂纹	既有面积阈值，还叠加了细长比、响应一致性筛选	单纯按面积筛选会把小裂纹漏掉、把大块污渍误判为裂纹；多个判据组合更稳

实测指标（60 张有裂纹 + 100 张无缺陷的 MT 数据集）：

指标	数值	说明
裂纹检测准确率	0.8662	整体判对比例
裂纹检测 F1	0.8235	综合精确率与召回率
平均定位 IoU	0.1438	偏弱，原因见 6.4 节
留出集 F1	0.7692	只在 157 张图上标定的参数，泛化后有所下降
工件识别 F1	0.9901	在合成数据集上
工件形状分类准确率	0.9767	在合成数据集上

要主动交代的两个局限：一是裂纹定位精度偏低（IoU 0.14），参数是在有限样本上标定的，换一批图需要重新标定；二是工件识别的数据集是程序合成的，真实产线光照、遮挡、反光情况复杂得多，结论不能直接外推。把这两句话说出来，比被追问出来要好得多。

6.7 关于任务 4 / 5 的常见提问

可能的提问	回答要点
为什么不用深度学习？	两点原因：一是课程任务书要求传统图像处理方法；二是产线上的缺陷样本往往只有几十张，训练深度学习模型数据量不够，而 Frangi 这类方法参数少、可解释、调参过程可以写清楚。
参数是怎么定的？	先用少量样本人工观察响应图，粗调阈值范围，再在数据集上网格搜索，按 F1 选优，最后固定下来。标定所用样本与留出验证样本是分开的。
IoU 只有 0.14，算法是不是不行？	要分开看：分类任务（有没有裂纹）指标正常，F1 0.82；定位任务（裂纹在哪里、多宽）确实偏弱。改进方向是把掩膜做膨胀或引入亚像素级宽度估计，这是明确的后续工作。
工件识别的置信度是怎么算的？	不是神经网络输出的概率，而是由形状逼近的相似度、圆度、顶点数量偏离程度等几个几何量组合成的分数，用来表达“这个分类有多像”，取值范围 0 到 1。
如果两个工件重叠在一起怎么办？	当前会当成一个连通区域，分不开。这是本方法的已知边界，重叠场景需要引入分水岭或模板匹配来分割。

第 7 章　任务 6：设备状态监测

7.1 这是一个“分类”问题

任务 6 的输入是机床（比如主轴）运行时的五个传感器读数，输出是这个设备当前处于哪种状态。这类问题在机器学习里叫有监督分类：有一批“已知答案”的数据用来学习，然后去判断新数据。

术语	在本项目里	通俗理解
特征（X）	温度、振动、电流、主轴转速、刀具磨损率	设备的体检指标
标签（y）	正常 / 磨损 / 故障	医生给的诊断结论
训练集	960 条已知状态的样本（三类各 320 条）	以前看过的病例
模型	随机森林分类器	学出来的“诊断经验”
预测	输入 120 条新读数 → 输出状态 + 置信度	给新病人下诊断

7.2 为什么先要“标准化”

五个特征的单位完全不一样：温度是几十到八十，转速是一千多，刀具磨损率是 0 到 1。如果不处理，计算距离或做分裂判断时，转速这个数字大的特征会主导一切，温度和磨损率等于白填。

标准化（StandardScaler）做的事：把每个特征减去自己的平均值、再除以自己的标准差，变换成“平均 0、波动 1”的新数值。这样五个特征就站在同一条起跑线上了。

```
标准化公式：   x' = ( x - 该特征的平均值 ) / 该特征的标准差

举例：主轴转速原始值 1500 rpm，均值 1480，标准差 40
      标准化后 = (1500 - 1480) / 40 = 0.5
      → 含义：比平均水平高半个标准差
```

必须记住的细节　标准化器 StandardScaler 是在训练集上拟合出来的，之后无论训练还是预测，都要用同一套均值和标准差去变换。如果每次预测都重新拟合，训练和预测的口径就不一致，模型表现会莫名其妙地变差。

7.3 从一棵决策树到一片森林

决策树：像玩“二十问”游戏。它每次挑一个特征和一个阈值，把数据分成两堆，比如“振动大于 2.8 mm/s 吗？”—大于的一堆继续往下分，直到每一堆里的样本状态基本一致，就贴上标签。

决策树的毛病：单独一棵树容易“背题”。训练数据里一点偶然的噪声它都当成规律记住，换一批新数据表现就会掉下来，这叫过拟合。

随机森林：干脆种很多棵树（本项目默认 `300` 棵），每棵树只看随机抽取的一部分样本和一部分特征，最后让所有树投票决定结果。单棵树的偏见被平均掉了，所以更稳。这就是“森林”比“一棵树”好的原因。

随机森林还有一个对我们很有用的副产品：它能告诉你每个特征被用来分裂的次数和带来的纯度提升，汇总起来就是这个特征的重要性。

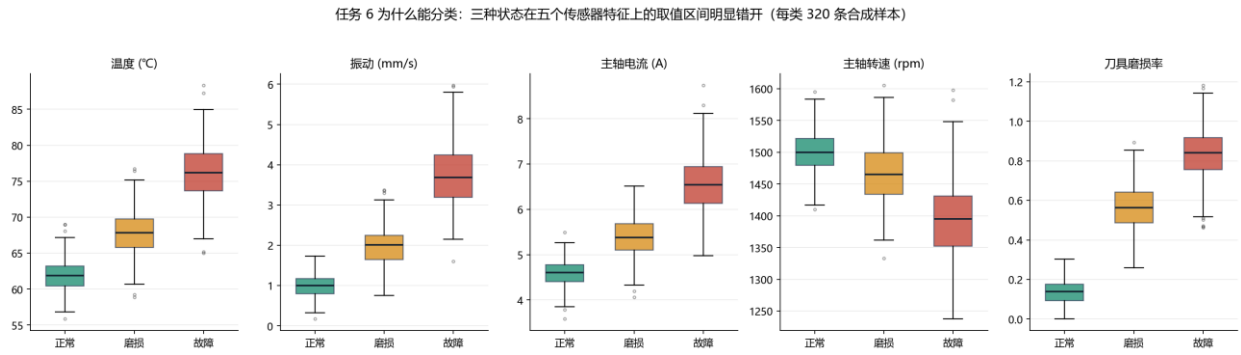


图 12　三类状态在五个特征上的取值范围：正常（绿）、磨损（黄）、故障（红）的区间明显错开，这就是分类可行的依据

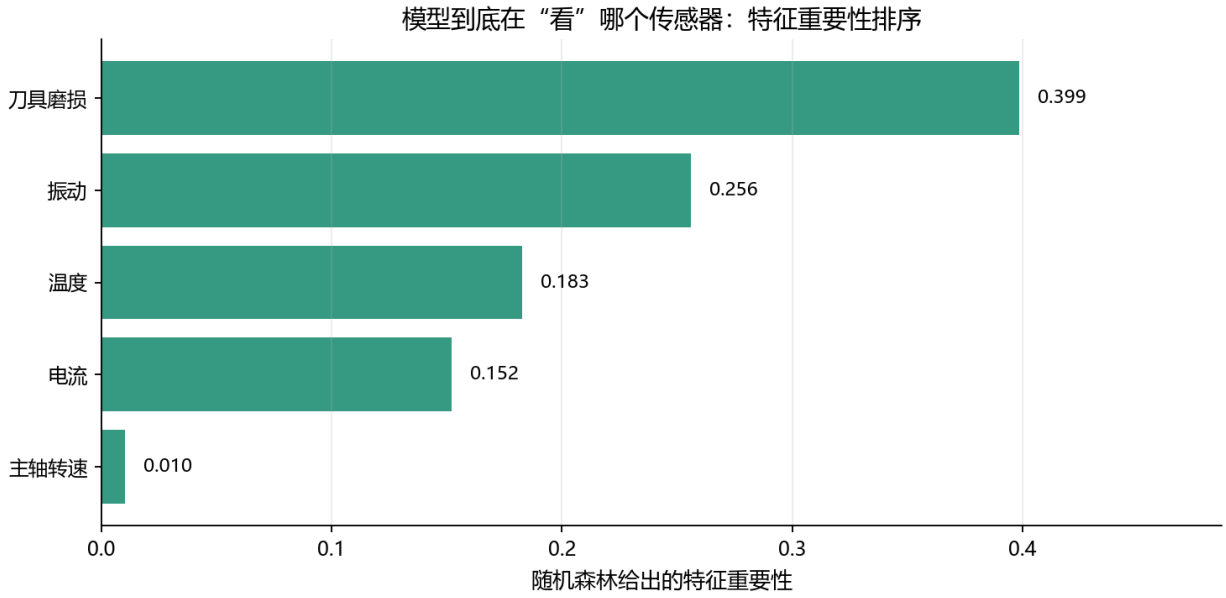


图 13　特征重要性排序：刀具磨损率贡献最大（约 `0.40`），其次是振动、温度、电流，主轴转速几乎没有信息量

图 11 怎么读：刀具磨损率的重要性接近 `0.4`，说明它是最能区分状态的特征—这符合机理常识：刀具磨了，状态当然首先反映在磨损量上。主轴转速的重要性只有 `0.01`，几乎不参与判断，说明这个特征在本数据集里对区分三种状态没有帮助。（注意：这不代表转速没用，只是说它在本数据集里没提供区分信息。）

7.4 模型表现怎么衡量

把 960 条数据按 8:2 分成训练集和验证集，用训练集学、用从没见过的验证集考。项目实测结果：

指标	数值	含义
准确率 Accuracy	0.9875	240 条验证样本里判对的比例
F1 宏平均（macro）	0.9874	对三类分别算 F1 再取平均，避免样本多的类别掩盖少数类
训练样本数	720	三类各 240 条
验证样本数	240	三类各 80 条

为什么要看“宏平均 F1”而不是只看准确率：假设三类样本数量差异很大，模型只要全部猜数量最多的那一类，准确率也可能很高，但对少数类完全失效。宏平均是对每一类单独算 F1 再平均，每一类的重要性一样，能暴露这种问题。

7.5 规则告警：模型之外的第二道闸

除了模型判断，项目还叠加了一组基于物理机理的阈值规则。比如温度超过某个值、振动超过某个值，就产生一条告警。这么做不是为了“让结果更好看”，而是因为两者关注的错误不一样：

	模型判断	规则告警
依据	从数据里学到的统计规律	明确的工程阈值，如“振动 > 3.1 mm/s”
优点	能捕捉多个特征组合起来的隐含模式	可解释、可追溯，工程师一眼能看懂
缺点	是黑箱，说不清“为什么”	阈值是固定的，换设备可能要重设
典型作用	综合判断当前状态	兜底：数据落在训练分布之外时仍然能报警

界面上的截图里可以看到这两者并存：综合状态显示“故障”，同时下方给出“主轴转速明显下降；振动超过警戒值 3.1 mm/s”的规则告警。

7.6 数据来源与已知局限

必须先说清楚的一件事：本项目的训练数据不是真实产线采集的，而是代码里按物理规律合成的（固定随机种子 42，可复现）。合成时给三个状态设定了不同的特征区间，所以模型能轻易学出很高的准确率——98.75% 这个数字只说明“算法流程是通的”，不能说明“在真实机床上也有 98.75% 的准确率”。

要把这个模块做到能真实使用，需要补的是：真实产线数据采集、人工标注、跨设备泛化验证、以及阈值按机型重新标定。这三件事在答辩里作为“后续工作”讲，是很自然的。

7.7 关于任务 6 的常见提问

可能的提问	回答要点
98.75% 的准确率可信吗？	算法流程上是可信的（训练集与验证集严格分开），但它反映的是“合成数据上的表现”，不是真实产线的表现。合成数据时三个状态的特征区间本来就是分开的，所以这个数字偏高是预期的。
为什么选随机森林，不用 SVM 或神经网络？	随机森林对特征量纲不敏感（我们仍做了标准化）、不需要复杂调参、能直接输出特征重要性、在小样本上不容易过拟合，而且训练只需秒级。对这种 5 个特征、3 个类别的问题，它是性价比最高的选择。
五个特征够吗？	对刀具磨损这一个场景够用，实际机床的状态监测通常还会加入声发射、功率、加工时长等。特征选择本身是一个可以展开的改进方向。
置信度是怎么来的？	随机森林里 300 棵树投票，比如 240 棵说是“故障”，置信度就是 240/300 = 0.8。界面上的 80.7% 就是这个含义。

第 8 章　任务 7：订单需求预测

8.1 时间序列和普通回归有什么不一样

普通回归问题里，样本之间是互相独立的：预测第 100 套房子的价格，跟第 99 套没有关系。时间序列完全不是这样——今天订单多，往往意味着明天也不少。这种“自己和自己相关”的特性叫自相关。

概念	含义	在本项目里怎么体现
趋势	长期上升或下降的方向	订单量整体缓慢上升，Holt-Winters 用“水平 + 趋势”两个分量建模
季节性	固定周期的重复波动	以 7 天为一个周期，周末订单明显下降，所以 Holt-Winters 设定周期为 7
随机扰动	没有规律的噪声	序列图里那些上上下下的锯齿
平稳性	统计特性不随时间改变	ARIMA 要求序列平稳，所以先做差分

8.2 三个模型分别在做什么

移动平均：最朴素的基准

用最近 k 个数据点的平均作为下一期的预测值。它不考虑趋势和季节，只是“平滑”掉噪声。把它放在项目里的作用不是因为它准，而是因为它是一个公认的基准：如果一个复杂模型连移动平均都赢不了，说明它没学到东西。

Holt-Winters：把水平和趋势、季节拆开

它假设序列由三部分组成，并分别跟踪：

```
预测值 = 水平项 + 趋势项 × h + 季节项

水平项：当前的基础值大概是多少
趋势项：每期上升或下降多少
季节项：相对于本月/本周的位置，应该加减多少

三个分量都用“指数平滑”更新：越新的数据权重越大，旧数据逐渐衰减。
```

本项目设定周期为 7（一周）。这个模型特别适合有明显周周期的订单数据。

ARIMA：先差分，再看自相关

ARIMA 由三个部分组成，名字里的三个字母就是它的三个参数：

- **AR（自回归）**：用过去若干期的数值来预测当前值，看“昨天的值对今天有多大影响”。

- **I（差分）**：把原始序列换成“相邻两天的差值”序列，消除趋势，让序列变平稳。这是 ARIMA 的前提条件。
- **MA（移动平均）**：用过去若干期的预测误差来修正当前预测，看“上次猜偏了多少”。

一句话比较三个模型：移动平均只看最近的“平均水位”；Holt-Winters 额外跟踪趋势和周期；ARIMA 则分析序列自身的相关结构。三种思路不同，所以结果会有差异，也就有比较的价值。

8.3 怎么判断哪个模型更好

项目里的自动模式先把历史数据切出一段做回测（使用最后一段已知数据作为“考卷”），让三个模型分别预测这一段，再和真实值对比算误差，选误差最小的那个，最后用完整历史数据预测未来。

指标	计算方式	特点
MAE 平均绝对误差	预测值与真实值之差的绝对值的平均	单位和原始数据一致，最容易理解
RMSE 均方根误差	误差平方的平均再开根号	对大误差惩罚更重，能暴露偶发的严重偏差
MAPE 平均绝对百分比误差	绝对误差 ÷ 真实值，再取平均（用百分数表示）	无量纲，可以跨不同量级的数据比较

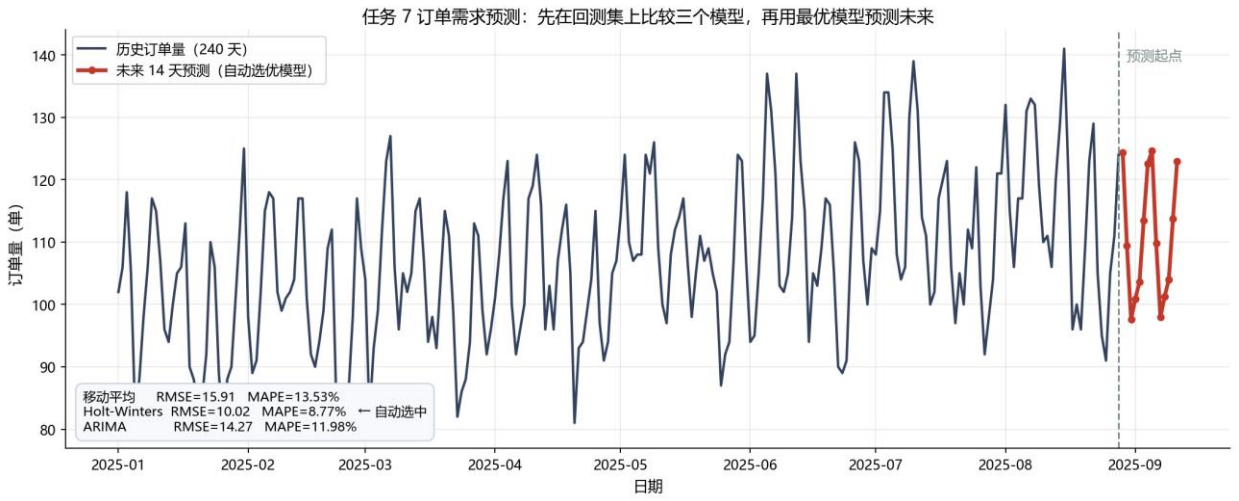


图 14　订单预测的真实运行结果：深色是 240 天历史订单，红色是未来 14 天预测，左下角是三个模型在回测集上的误差对比

本项目这一轮的实测结果：

模型	MAE	RMSE	MAPE	结论
移动平均	14.1904	15.907	13.53%	基准模型，误差最大
Holt-Winters	9.2211	10.0162	8.77%	三项指标全面最优，被自动选中
ARIMA	12.6461	14.2683	11.98%	介于两者之间

为什么 Holt-Winters 赢了：因为这份订单数据本身带有明显的周周期（周末下降）和缓慢上升趋势，而 Holt-Winters 恰好是专门为“趋势 + 周期”设计的。移动平均完全没有周期概念，所以最差；ARIMA 我们用的是固定阶数，没有针对周周期做季节性扩展（完整的做法是 SARIMA），所以略逊一筹。这个解释能体现你理解模型差异，而不只是报一个数字。

8.4 数据来源与已知局限

和任务 6 一样，本项目的订单数据也是代码合成的（固定种子，具有趋势、周周期和随机扰动）。所以上面这组数字说明的是“算法选择流程正确”，而不是“这套模型能预测你们公司的真实订单”。换成真实数据时，需要重新检查周期性、处理缺失值与促销等异常点，并延长回测区间。

8.5 关于任务 7 的常见提问

可能的提问	回答要点
为什么不让三个模型都输出结果让人选？	自动模式仍然会把三个模型的指标全部列出来（见界面左下角），只是额外给出一个推荐结论。给结论同时给依据，使用者可以推翻推荐。
预测 14 天为什么比预测 90 天准？	时间序列预测的误差会随预测步长累积，越远的未来越不确定。所以要给出预测区间而不是单点值，这也是后续可以补的功能。
为什么不直接上 LSTM？	数据量只有 240 天，深度学习模型参数多、容易过拟合，而且训练和调参成本高。对这种规模的序列，统计方法通常是更稳的选择。
MAPE 8.77% 算好吗？	在订单预测里，10% 以内通常被认为可以用于排产参考，20% 以上就只能当趋势看了。但要强调这是合成数据的成绩。

第 9 章　工程实现：前后端是怎么接起来的

9.1 Django 在这个项目里承担了什么

Django 是一个 Python 的网站开发框架。它把自己拆成几块职责，本项目用到的部分如下：

文件	职责	本项目的具体内容
models.py	定义数据表结构	地图节点、道路、障碍、AGV、运输任务、调度记录、检测记录、诊断记录、预测记录
serializers.py	把数据库对象转成 JSON，也把 JSON 转回对象	每个模型对应一个序列化器，同时负责字段校验
views.py	处理 HTTP 请求，调用算法，返回结果	项目里最“厚”的文件之一，负责参数校验与异常转错误信息
services.py	纯粹的算法实现，不涉及网络和数据库结构	调度、路径、视觉、诊断、预测的全部核心逻辑都在这里
urls.py	路由表：什么地址交给哪个视图	根路由把 /api/、/api/vision/ 等前缀分配给不同应用
admin.py	后台管理界面	可在 http://127.0.0.1:8000/admin/ 直接查看和修改数据

一个重要的设计习惯：算法写在 services.py 里，视图只负责“收参数、调算法、返回结果”。这样算法可以被命令行脚本、测试代码、后台任务复用，也更容易单独测试。如果你以后要改算法，请改 services.py，不要改 views.py。

9.2 主要接口清单

这张表是接口的总账。答辩前把它看一遍，别人问到任何一个功能，你都能说出对应的地址。

方法	地址	作用
GET	/api/overview/	仪表盘统计：车队利用率、任务分布、最近调度记录
GET	/api/map/	地图全部数据：节点、道路、障碍、车辆位置、活动路线
POST	/api/path-planning/	给定起点终点，规划最短路径
GET/POST/…	/api/nodes/　/api/edges/ /api/obstacles/	地图元素的增删改查
GET/POST/…	/api/agvs/	AGV 增删改查；/api/agvs/summary/ 返回状态分布
GET/POST/…	/api/tasks/	运输任务增删改查；/{id}/start/、/complete/、/cancel/ 改变任务状态

方法	地址	作用
GET	/api/dispatches/	调度明细记录（每条任务由哪台车执行、路线、里程）
GET	/api/schedule-runs/	调度优化记录与指标（makespan、改进率、迭代次数）
POST	/api/schedules/single/	单台 AGV 多任务顺序优化（任务 1）
POST	/api/schedules/batch/	多台 AGV 联合调度（任务 2）
POST	/api/vision/crack/	裂纹检测，上传图片或指定内置样例（任务 4）
POST	/api/vision/detect/	工件识别（任务 5）
GET	/api/vision/records/	视觉检测历史记录
POST	/api/equipment/diagnose/	设备状态诊断（任务 6）
GET	/api/equipment/model/	模型指标与特征重要性
POST	/api/forecast/predict/	订单需求预测（任务 7）
GET	/api/forecast/runs/	预测历史记录

9.3 前端：8 个页面与一个轮询

前端一共 8 个页面，每个页面对应左侧菜单的一项，也对应 src/views/ 下的一个 .vue 文件。所有页面的公共外壳（左侧菜单、顶部标题、提示气泡、自动刷新）都写在 src/App.vue 里。

```
// frontend/src/App.vue —— 每 8 秒自动同步一次数据
onMounted(() => {
  loadAll()                                          // 首次进入立即拉一次
  pollTimer = setInterval(() => loadAll(true), 8000)   // 之后每 8 秒拉一次
})

async function loadAll(silent) {
  const [summary, fleet, taskList, dispatchList, graph] = await Promise.all([
    api.overview(), api.agvs(), api.tasks(), api.dispatches(), api.map(),
  ])   // 五个请求并行发出，等全部返回后一起更新界面
  overview.value = summary; agvs.value = fleet; tasks.value = taskList; ...
}
```

这种“定时轮询”的写法简单可靠，代价是会产生大量重复请求：每次都会把整个地图（约 78 KB）和全部任务（约 19 KB）重新拉一遍。这是本项目可以优化的地方，也是答辩里“后续改进方向”的好素材。

9.4 界面一览

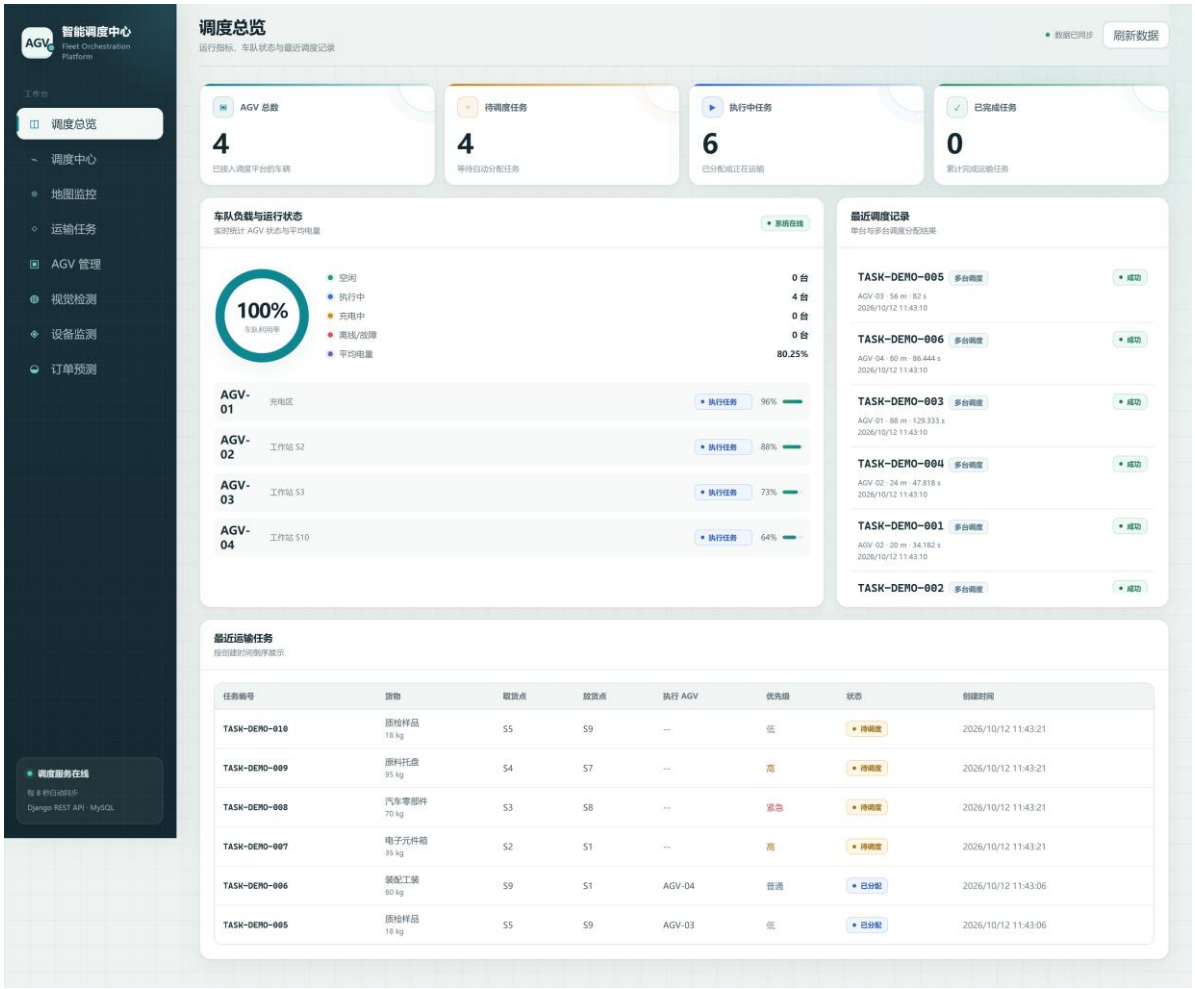


图 15　调度总览页：车队状态、电量、任务分布与最近调度记录

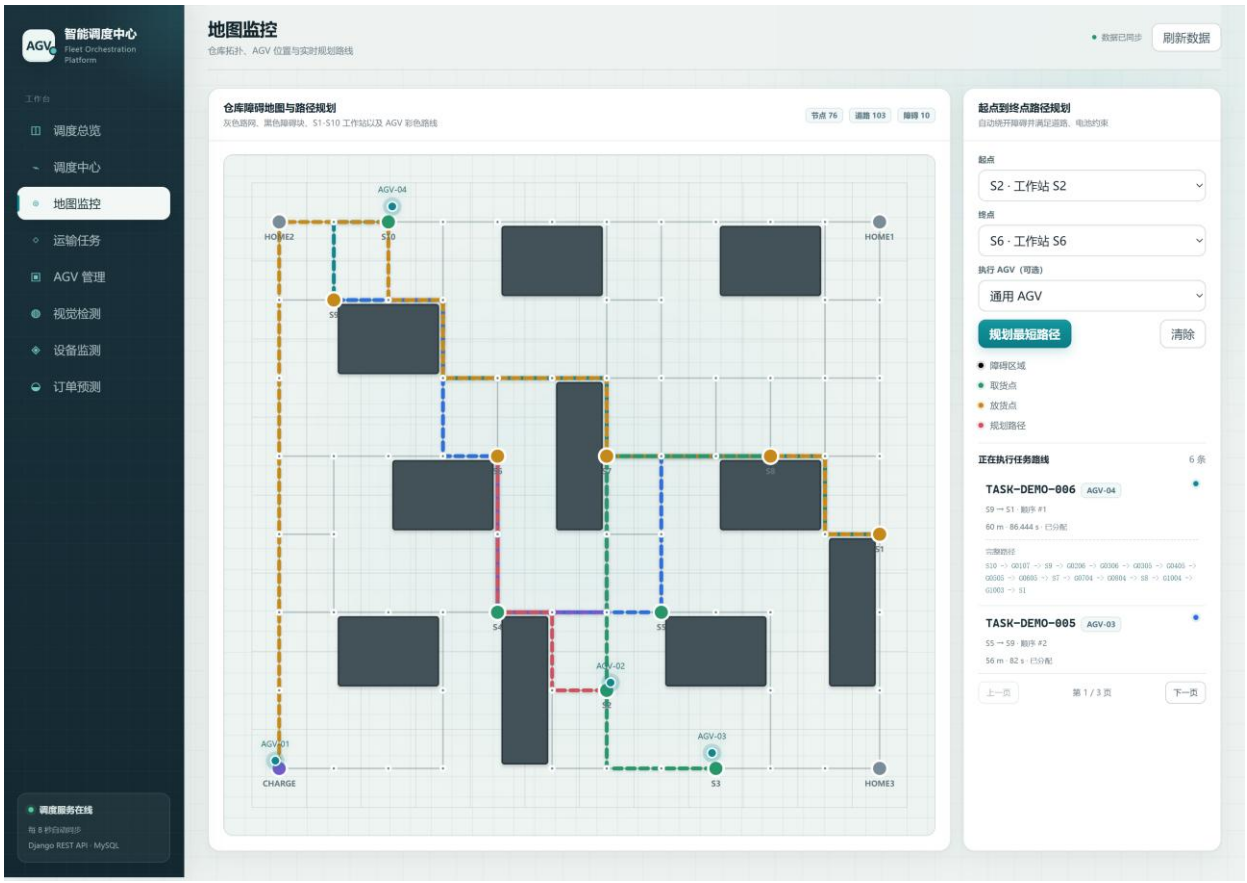


图 16　地图监控页：多任务彩色路线叠加显示，右侧可查看每条执行中任务的完整节点序列

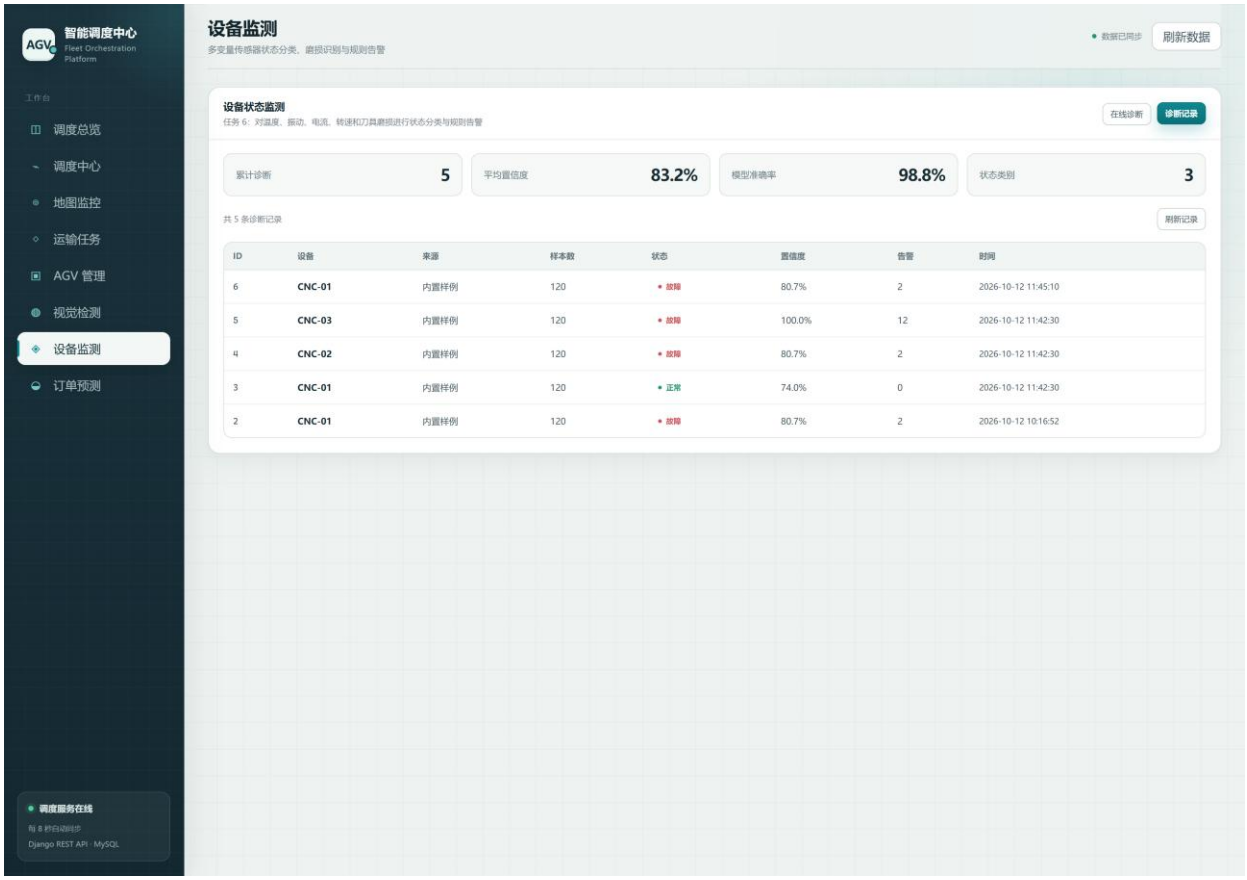


图 17　设备监测的诊断记录页：每次诊断的状态、置信度、来源与时间都落库保存

9.5 文件上传与图片存储

视觉模块需要接收用户上传的图片，这在 Django 里走的是 multipart 表单，和普通 JSON 请求不同：

29. 前端把文件塞进 FormData 对象，而不是 JSON。api.js 里专门给上传请求留了一个分支。
30. Django 通过 request.FILES 取到文件，而不是 request.data。
31. 文件被保存到 media/ 目录，数据库里只记录文件路径。这样数据库不会被图片撑大。
32. 页面上要显示图片时，访问 /media/... 路径；开发模式下由 Django 直接托管。

一个部署注意点：media/ 目录已经在 .gitignore 里被忽略，所以它不会进版本库；部署到服务器时要注意单独迁移这个目录，否则历史检测记录里的图片会全部失效。

9.6 如果要部署到服务器

- 把 DJANGO_DEBUG 改成 False，并修改 DJANGO_SECRET_KEY 和数据库密码——现在的默认值只适合本机演示。
- 前端执行 npm run build 生成静态文件（本项目实测产物约 147 KB JS + 25 KB CSS），交给 Nginx 托管。
- Nginx 把 /api 反向代理到 Django，把 /media 指向图片目录。
- 把 opencv-python 换成 opencv-python-headless，因为服务器通常没有图形界面，headless 版本体积更小。
- 当前所有接口默认允许匿名访问（AllowAny），是为了课程演示方便；真正上线要加登录鉴权。

第 10 章　测试、数据与复现

10.1 自动化测试覆盖了什么

项目共有 40 个自动化测试，全部通过（实测耗时约 12 秒）。测试用的是一个独立的临时数据库，跑完会自动销毁，不会污染你的演示数据。

测试模块	覆盖内容
SchedulingDecision	Dijkstra 路径生成、单台自动选车、多台批量分配、多车路线与避碰等待、任务开始/完成后的车辆位置与电量更新、仪表盘与地图接口
ManufacturingVision	裂纹检测算法、工件识别算法、数据集访问、批量评估命令、数据集生成命令
EquipmentMonitoring	诊断服务与接口、规则告警、模型指标接口
DemandForecast	三种预测模型、自动选优逻辑、预测接口与记录接口

```
# 跑全部测试
python manage.py test -v 2

# 只跑某一个模块
python manage.py test SchedulingDecision -v 2
python manage.py test ManufacturingVision -v 2
python manage.py test EquipmentMonitoring -v 2
python manage.py test DemandForecast -v 2

# 前端构建检查
cd frontend
npm run build
```

测试需要 MySQL　测试会临时创建并销毁一个名为 test_agv 的数据库，所以 MySQL 必须先启动，而且账号要有建库权限。数据库没开时测试会直接报连接错误，不是代码问题。

10.2 从零复现整套环境

33. 安装 Python 3.10 以上版本，建议用 conda 建一个独立环境（本项目用的是名为 pmp 的环境）。
34. 在项目根目录执行 pip install -r requirements.txt 安装全部依赖。
35. 安装并启动 MySQL，创建数据库 agv（字符集用 utf8mb4）。
36. 执行 python manage.py migrate 建表。
37. 执行 python manage.py seed_demo --reset 初始化演示数据（会清空已有业务数据）。
38. 执行 python manage.py runserver 127.0.0.1:8000 启动后端，浏览器访问 http://127.0.0.1:8000/api/ 能看到接口列表。

39. 进入 frontend 目录执行 npm install 和 npm run dev 启动前端，浏览器访问 http://127.0.0.1:5173。
40. 用 admin / Admin123! 登录 http://127.0.0.1:8000/admin/ 可以查看后台数据。

初始化脚本会生成：76 个地图节点、103 条道路、10 个障碍区域、4 台 AGV、6 个待调度演示任务。

10.3 几个会让演示翻车的坑

坑	现象	怎么处理
seed_demo --reset 会清空数据	演示前想恢复数据，执行后自己之前演示出来的调度记录、检测记录全没了	这是预期行为，重置会同时清空 ScheduleRun 等记录表。所以要在演示前重置，不要演示后重置
后端日志刷屏	浏览器标签页放后台，日志每隔一分钟刷出一组请求	这是前端的 8 秒轮询被浏览器后台节流了，属于正常现象。不用管，关掉多余网页即可
任务列表的筛选参数不生效	访问 /api/tasks/?status=pending 返回的还是全部任务	这是一个真实的代码缺陷：Django REST Framework 的过滤后端的配置里缺少 DjangoFilterBackend，导致各视图里写的 filterset_fields 没有生效。修复方法：安装 django-filter 并在 DEFAULT_FILTER_BACKENDS 中加上 django_filters.rest_framework.DjangoFilterBackend，同时把 django_filters 加入 INSTALLED_APPS
任务不能直接完成	调用 /complete/ 报 400「只有执行中的任务可以完成」	必须先调用 /start/ 把任务转为执行中，再调用 /complete/
前序任务未完成	调度后想开始第二个任务，报「存在未完成的前序任务，请按优化顺序执行」	按调度给出的顺序依次 start → complete，这是设计上的约束，不是错误

第 11 章　答辩准备

11.1 五分钟演示脚本

答辩演示最怕的不是讲不出东西，而是手忙脚乱、点错页面。下面这份脚本按顺序做完，刚好能覆盖七个任务，而且每一步都有可见的结果。建议提前走两遍，把时间掐准。

顺序	操作	要讲的一句话	预计耗时
0	启动前后端，确认左侧显示「调度服务在线」	系统由 Django 后端、MySQL 数据库和 Vue 前端三部分组成	20 秒
1	打开「调度总览」	这是运行总览：车队利用率、平均电量、任务分布和历史调度记录	20 秒
2	打开「地图监控」，选起点 S2、终点 S6，点击规划	地图用 Dijkstra 求最短路径，红色是规划结果，它会自动绕开障碍	40 秒
3	打开「调度中心」，勾选 4 个任务，点单台优化	单台调度用状态压缩动态规划求最优执行顺序，完工时间从 350.3 秒降到 306.6 秒，改进 15.33%	60 秒
4	点「执行多台 AGV 联合调度」	多台调度用变邻域搜索同时优化任务分配和车内顺序，并做节点时间窗避碰	60 秒
5	打开「视觉检测」，选裂纹样例，点开始检测	裂纹检测用高斯滤波加多尺度黑脊响应，5 毫秒出结果，F1 是 0.82	50 秒
6	切到工件识别，点开始检测	工件识别输出形状、位置和置信度，F1 是 0.99	30 秒
7	打开「设备监测」，点开始诊断	随机森林结合机理规则判断设备状态，输出置信度和特征重要性	40 秒
8	打开「订单预测」，点开始预测	自动在回测集上比较三个模型，Holt-Winters 的 MAPE 最低被自动选中	40 秒
9	回到调度总览收尾	整套系统的数据和历史记录都持久化在 MySQL 中，可追溯	20 秒

演示前必做　先执行 `python manage.py seed_demo --reset` 恢复演示数据，再按上面的顺序自己走一遍。另外，多台调度的改进率对任务数量很敏感，如果现场只有 4 个任务和 4 台空闲车，很可能显示 0%，建议提前准备 6 个以上任务、并只让 3 到 4 台车可用。

11.2 高频问题速查

下面 20 个问题是这类项目最容易被问到的。每条都给了回答的骨架，请用自己的话讲出来，不要背句子。

问题	回答骨架
这个项目做了什么？	一句话：把仓库抽象成图，用优化算法解决 AGV 的任务分配与顺序问题，再叠加三个智能制造智能分析模块

问题	回答骨架
为什么要做这个题目？	与课程七个任务对应；AGV 调度是智能制造的典型场景；任务书要求的算法（最短路径、动态规划、机器学习、时间序列）覆盖了课程主要知识点
整个系统分几层？	四层：Vue 前端 → REST 接口 → Django 应用与算法 → ORM → MySQL
为什么前后端分离？	算法只写一遍，多种客户端都能调用；前端改界面不影响算法；便于分工并行开发
调度算法用了哪些？	Dijkstra（路径）、状态压缩动态规划与 2-opt（单台顺序）、最优插入与变邻域搜索（多台分配）、节点时间窗（避碰）
为什么路径规划不能用 BFS？	BFS 只适合边权相等的情况，本项目边权是距离，不等
动态规划的状态是什么？	(已完成任务集合，最后完成的任务)，集合用二进制位掩码表示
为什么任务超过 10 个就换算法？	$n \cdot 2^n$ 的状态数在 n 变大后会失控，改用最近邻加 2-opt，牺牲精确性换时间
多台调度怎么避免撞车？	每个节点登记占用时间窗，冲突时插入等待时间，等待计入目标函数
目标函数包括哪些项？	完工时间、使用的车辆数、避碰等待、总里程、优先级加权完成时间，加权求和
为什么改进率有时是 0？	任务数不超过车辆数时，贪心基准和优化算法会得到同一个解，这恰恰说明该规模下贪心已经足够好
裂纹检测的原理？	高斯滤波去噪，多尺度 Frangi 黑脊响应突出细而暗的线状结构，阈值分割后按连通域形状筛选
为什么不用边缘检测算子？	边缘算子分不清裂纹边缘和工件轮廓、刀痕边缘；黑脊响应带有细和暗的物理约束，更针对裂纹
IoU 只有 0.14 怎么解释？	分类指标正常（F1 0.82），定位指标偏弱；原因是掩膜偏保守；改进方向是形态学膨胀或亚像素宽度估计
设备监测为什么用随机森林？	小样本不敏感、能输出特征重要性、训练快、不容易过拟合；配合机理规则告警互补
训练数据是真实的吗？	不是，是按物理规律合成的，固定种子可复现。所以高准确率只说明流程正确，不能外推到真实产线
为什么先做标准化？	五个特征量纲差异巨大，不标准化会让大数值特征主导判断
三个预测模型怎么比较？	在回测集上统一算 MAE、RMSE、MAPE，选最小者，再用完整历史预测未来
Holt-Winters 为什么胜出？	订单数据有周周期和趋势，Holt-Winters 专门建模这两个分量；移动平均没有周期概念，ARIMA 未做季节性扩展
项目的不足和后续工作？	接口没有鉴权；用轮询而不是 WebSocket；视觉模块定位精度低、数据集是合成的；任务列表的过滤器配置有缺陷待修；可增加真实数据接入

11.3 薄弱环节与应对原则

有几处地方如果被追问会觉得心虚，提前想好原则就不慌：

- **数据是合成的**：任务 6 和任务 7 的数据由代码生成。不要回避，主动说明“可复现，用于验证算法流程”，并指出接入真实数据需要做哪些工作。
- **视觉定位指标偏低**：坦白 IoU 偏低，同时把分类指标和定位指标分开讲，给出明确的改进思路。
- **没有鉴权、没有实时推送**：说明这是课程演示的取舍，并说清生产环境要加什么。

- **过滤器配置缺陷**：如果被问到筛选功能，如实说明这是已定位但待修复的配置问题，并给出修复方案—能定位自己的缺陷，本身就是加分项。
- **优化算法不是自己原创**：Dijkstra、动态规划、变邻域搜索都是经典算法。答辩时应该强调“为什么在这个问题上选它、怎么把实际约束（电量、载重、避碰）建模进去”，而不是声称算法是自己发明的。

11.4 小组分工与材料清单

代码提交记录显示有三个账号参与开发，而小组是四个人。建议这样分工，让每个人在答辩里都有明确的一块：

角色	负责内容	答辩时负责讲
A（组长）	整体进度、调度内核、集成与提交	系统架构、任务 1/2/3 与整体演示
B	视觉模块	任务 4/5：算法原理、指标与本模块局限
C	设备监测与订单预测	任务 6/7：模型选择与数据来源说明
D	文档、测试与演示材料	测试覆盖、项目复现流程、组内过程管理

需要准备的材料清单：

- 课程设计报告（需求分析、总体设计、各任务详细设计、实验结果、测试、总结与展望）
- 答辩演示 PPT
- 演示视频（建议 3 到 5 分钟，覆盖七个任务）
- 源代码与 README（已包含完整的功能清单、接口清单、启动步骤）
- 测试报告（40 个测试的覆盖说明与实际结果）
- 大语言模型使用说明（课程明确要求，需注明用了哪些工具、用在哪些环节、自己做了哪些修改与校对）

最后一句实话　答辩老师最看重的是“你是不是真的懂自己在做什么”。把这本材料里的图和数字都亲手复现一遍，比背下任何话术都有用。

附录 A　术语速查

术语	英文 / 缩写	一句话解释
节点 / 边	Node / Edge	图里的点和线；本项目里节点是可停车位置，边是道路
图搜索	Graph Search	在由点和线组成的网络里找路径
最短路径	Shortest Path	总权重最小的那条路径
优先队列	Priority Queue	每次都能快速取出最小元素的容器，Dijkstra 的核心数据结构
邻接表	Adjacency List	用“每个节点对应一个邻居列表”的方式存储图
makespan	最大完工时间	一批任务从开始到全部结束所花的时间
贪心算法	Greedy	每一步都选眼前最优，不保证全局最优
动态规划	Dynamic Programming	把问题拆成子问题，记住子问题答案避免重复计算
状态压缩	Bitmask DP	用二进制位表示集合，把集合状态压进一个整数
2-opt	—	通过翻转路径片段来缩短总长的局部搜索方法
变邻域搜索	VNS	轮流使用多种邻域变换来改进解的元启发式算法
启发式 / 元启发式	Heuristic / Metaheuristic	不保证最优、但能在可接受时间内给出足够好解的算法
时间窗	Time Window	某资源在某段时间内被占用，用于多车避碰
灰度图	Grayscale	只有明暗没有颜色的图像，可看作一个二维数组
滤波	Filtering	用邻域像素加权平均来平滑或增强图像
Hessian 矩阵	—	由二阶偏导数组成的矩阵，描述局部曲率
黑脊响应	Frangi Vesselness	衡量某点是否属于“细而暗的线状结构”的分数
阈值分割	Thresholding	按一个数值把图像分成目标和背景两部分
连通域	Connected Component	相互连通的像素组成的一块区域
形态学运算	Morphology	腐蚀、膨胀、开、闭等基于形状的图像操作
多边形逼近	approxPolyDP	用尽量少的直线段去逼近轮廓形状
Otsu 阈值	大津法	自动选择让前景背景差异最大的分割阈值
交并比	IoU	两块区域的重叠面积除以合并面积，衡量定位是否准确
精确率 / 召回率	Precision / Recall	判准的比例 / 找全的比例
F1 分数	F1 Score	精确率与召回率的调和平均
标准化	Standardization	把特征变换成均值 0、标准差 1
随机森林	Random Forest	很多棵随机化的决策树投票得出结果
特征重要性	Feature Importance	衡量每个特征对判断结果的贡献大小
过拟合	Overfitting	模型把训练数据的噪声当成规律记住，换新数据就失效
回测	Backtesting	用历史数据模拟预测并检验误差
MAE / RMSE / MAPE	—	三种预测误差指标：平均绝对误差、均方根误差、平均绝对百分比误差
自相关	Autocorrelation	序列当前值和过去值之间的相关性
差分	Differencing	用相邻两期数值之差构成新序列，用于消除趋势
ORM	Object-Relational Mapping	用 Python 类操作数据库表，不用写 SQL
DRF	Django REST Framework	Django 上开发 REST 接口的扩展库

术语	英文 / 缩写	一句话解释
序列化器	Serializer	负责对象与 JSON 之间的相互转换和字段校验
跨域	CORS	浏览器对“不同源”请求的限制；本项目用 django-cors-headers 放开
轮询	Polling	前端定时反复请求后端获取最新数据

附录 B　关键代码位置索引

想找什么	去哪里看
地图数据结构、AGV、任务的数据表	SchedulingDecision/models.py
地图构建、Dijkstra 路由器	SchedulingDecision/services.py 的 build_graph() 与 GraphRouter
单台 AGV 顺序优化	SchedulingDecision/services.py 的 optimize_single_solution()
多台 AGV 联合优化	SchedulingDecision/services.py 的 optimize_multi_solution()
基准算法与初始解构造	SchedulingDecision/services.py 的 build_baseline_solution() 与 build_insertion_solution()
多车时间窗避碰	SchedulingDecision/services.py 的方案评估函数
演示数据初始化	SchedulingDecision/management/commands/seed_demo.py
裂纹检测算法	ManufacturingVision/services.py 的 analyze_crack() 与 _vesselness_map()
工件识别算法	ManufacturingVision/services.py 的 analyze_workpiece() 与 _classify_shape()
视觉数据集管理	ManufacturingVision/datasets.py
视觉批量评估命令	ManufacturingVision/management/commands/evaluate_vision.py
设备诊断与合成数据	EquipmentMonitoring/services.py
订单预测三个模型	DemandForecast/services.py
接口路由	AGV/urls.py 与各应用的 urls.py
全局配置（数据库、跨域、分页）	AGV/settings.py
前端页面	frontend/src/views/ 下的 8 个 .vue 文件
前端请求封装	frontend/src/api.js
前端整体布局与轮询	frontend/src/App.vue

附录 C　核心指标汇总

模块	指标	实测值	数据来源
路径规划	地图规模	76 节点 / 103 道路 / 10 障碍	地图接口实时统计
路径规划	示例路线	CHARGE → HOME1，19 个节点，72 米	真实运行结果

模块	指标	实测值	数据来源
单台调度	完工时间改进	350.3 s → 306.6 s（改进 15.33%）	4 个任务的真实调度记录
单台调度	搜索迭代次数	64 次	同上
多台调度	完工时间改进	149.64 s → 131.71 s（改进 6.44%）	6 个任务配 4 台车的真实调度记录
多台调度	搜索迭代次数	3035 次	5 个任务配 4 台车的真实调度记录
多台调度	避碰等待	11.6 s（5 任务场景）	同上
视觉 / 任务 4	检测准确率	0.8662	MT 数据集 60 张有裂纹 + 100 张无缺陷
视觉 / 任务 4	F1 / 精确率 / 召回率	0.8235 / 0.7903 / 0.8596	同上
视觉 / 任务 4	平均定位 IoU	0.1438（偏弱）	同上
视觉 / 任务 4	单张耗时	约 5 毫秒	接口实测
视觉 / 任务 5	检测 F1	0.9901	合成工件数据集 120 张
视觉 / 任务 5	形状分类准确率	0.9767	同上
设备监测	准确率 / F1 宏平均	0.9875 / 0.9874	合成数据 720 训练 + 240 验证
设备监测	首要特征	刀具磨损率（重要性约 0.40）	随机森林特征重要性
订单预测	最优模型	Holt-Winters	自动回测选优
订单预测	MAE / RMSE / MAPE	9.2211 / 10.0162 / 8.77%	240 天历史 + 14 天预测
整体	自动化测试	40 项全部通过（约 12 秒）	manage.py test
整体	前端构建体积	JS 147 KB / CSS 25 KB	npm run build